

UCS 2511 - Network Lab

Exercise 01 - Network Commands

Name: Aathish Sivasubrahmanian

Reg No: 3122215001001

Semester: V

Aim:

To Learn and understand the use of commands like tcpdump, netstat, ifconfig, nslookup and traceroute, ping .

1. tcpdump

- Description:

Tcpdump prints out a description of the contents of packets on a network interface that matches the boolean expression specified.

- Syntax: tcpdump [options]

Table 1: Options and their meanings

Option	Meaning
<code>-D</code> <code>-i <i>hinterf</i></code> <code>-c <i>count</i></code>	List of interfaces tcpdump can track Listen on specified interface Exit after receiving 'count' packets

- Output:

Figure 1: Listing Interfaces

```
$ tcpdump -D
1.eth0 [Up, Running, Connected]
2.any (Pseudo-device that captures on all interfaces) [Up, Running]
3.lo [Up, Running, Loopback]
4.bluetooth-monitor (Bluetooth Linux Monitor) [Wireless]
5.nflog (Linux netfilter log (NFLOG) interface) [none]
6.nfqueue (Linux netfilter queue (NFQUEUE) interface) [none]
7.dbus-system (D-Bus system bus) [none]
8.dbus-session (D-Bus session bus) [none]

$
```

Figure 2: tcpdump for a particular interface

```

File Actions Edit View Help Run Terminal Help
$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
$

```

Figure 3: Listing 'count' packets

```

$ sudo tcpdump -i eth0 -c 3
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
$

```

2. netstat

- Description:
Print network connections, routing tables, interface statistics, masquerade connections, and multicast memberships.
- Syntax: netstat [options]

Table 2: Option and their meanings

Option	Meaning
-a	Show both listening and non-listening sockets. Display summary statistics for each protocol. All TCP ports
-s	All UDP ports
-t	Listening ports
-u	
-l	

- Output:

Figure 4: Output of netstat

```

$ netstat
Active Internet connections (w/o servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp        0      0 192.168.64.3:bootpc    192.168.64.1:bootps    ESTABLISHED

Active UNIX domain sockets (w/o servers)
Proto RefCnt Flags       Type       State         I-Node  Path
unix   3      [ ]          STREAM     CONNECTED   19792    /run/user/1000/bus
unix   3      [ ]          STREAM     CONNECTED   20684    /run/user/1000/at-spi/bus_0
unix   3      [ ]          STREAM     CONNECTED   15353    /run/dbus/system_bus_socket
unix   3      [ ]          STREAM     CONNECTED   89811    /run/systemd/journal/stdout
unix   3      [ ]          STREAM     CONNECTED   18929    /run/user/1000/bus
unix   3      [ ]          STREAM     CONNECTED   20662
unix   3      [ ]          STREAM     CONNECTED   20606
unix   3      [ ]          STREAM     CONNECTED   17810
unix   3      [ ]          DGRAM      CONNECTED   17019
unix   3      [ ]          DGRAM      CONNECTED   15025
unix   3      [ ]          STREAM     CONNECTED   89896
unix   3      [ ]          STREAM     CONNECTED   94209    /run/systemd/journal/stdout
unix   3      [ ]          STREAM     CONNECTED   18878    /run/systemd/journal/stdout
unix   3      [ ]          STREAM     CONNECTED   14810    /run/systemd/journal/stdout
unix   3      [ ]          STREAM     CONNECTED   322879
unix   3      [ ]          STREAM     CONNECTED   94417
unix   3      [ ]          STREAM     CONNECTED   19854    /run/user/1000/bus
unix   3      [ ]          STREAM     CONNECTED   21505    /run/user/1000/bus
unix   2      [ ]          DGRAM      CONNECTED   15234
unix   2      [ ]          DGRAM      CONNECTED   15224
unix   3      [ ]          STREAM     CONNECTED   92119
unix   3      [ ]          STREAM     CONNECTED   93117
unix   3      [ ]          SEQPACKET  CONNECTED   92071
unix   3      [ ]          STREAM     CONNECTED   18989

```

Figure 5: Statistics for protocols

```

CODE ENDS
$ netstat -s      END START
Ip:
    Forwarding: 2
    22292 total packets received
    12 with invalid addresses
    0 forwarded
    0 incoming packets discarded
    22263 incoming packets delivered
    11999 requests sent out
    20 dropped because of missing route
Icmp:
    1 ICMP messages received
    0 input ICMP message failed
    ICMP input histogram:
        destination unreachable: 1
    16 ICMP messages sent
    0 ICMP messages failed
    ICMP output histogram:
        destination unreachable: 16
IcmpMsg:
    InType3: 1
    OutType3: 16
Tcp:
    30 active connection openings
    0 passive connection openings
    2 failed connection attempts
    14 connection resets received
    0 connections established
    22068 segments received
    11810 segments sent out
    64 segments retransmitted
    0 bad segments received
    3 resets sent
Udp:
    156 packets received
    16 packets to unknown port received
    0 packet receive errors
    187 packets sent
    0 receive buffer errors
    0 send buffer errors
    IgnoredMulti: 77
UdpLite:
TcpExt:
    4 TCP sockets finished time wait in fast timer
    43 delayed acks sent
    Quick ack mode was activated 8 times

```

Figure 6: Listening ports

```

$ netstat -l
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp6      0      0 fe80::e07:dhcpx6-client [::]:*
raw6      0      0 [::]:ipv6-icmp          [::]:*                  7

Active UNIX domain sockets (only servers)
Proto RefCnt Flags       Type       State      I-Node  Path
unix  2      [ ACC ] STREAM    LISTENING   14909    /tmp/.X11-unix/X0
unix  2      [ ACC ] STREAM    LISTENING   20568    /tmp/ssh-TOD0Uu7xEbUZ/agent.792
unix  2      [ ACC ] STREAM    LISTENING   19586    /tmp/.ICE-unix/792
unix  2      [ ACC ] STREAM    LISTENING   18700    /run/user/1000/systemd/private
unix  2      [ ACC ] STREAM    LISTENING   1355     /run/systemd/private
unix  2      [ ACC ] STREAM    LISTENING   15248    /run/user/1000/bus
unix  2      [ ACC ] STREAM    LISTENING   1357     /run/systemd/userdb/io.systemd.DynamicUser
unix  2      [ ACC ] STREAM    LISTENING   1358     /run/systemd/io.system.ManagedOOM
unix  2      [ ACC ] STREAM    LISTENING   15249    /run/user/1000/gnupg/S.dirmngr
unix  2      [ ACC ] STREAM    LISTENING   15251    /run/user/1000/gcr/ssh
unix  2      [ ACC ] STREAM    LISTENING   15253    /run/user/1000/keyring/control
unix  2      [ ACC ] STREAM    LISTENING   15255    /run/user/1000/gnupg/S.gpg-agent.browser
unix  2      [ ACC ] STREAM    LISTENING   1372     /run/lvm/lvmpolld.socket
unix  2      [ ACC ] STREAM    LISTENING   15257    /run/user/1000/gnupg/S.gpg-agent.extra
unix  2      [ ACC ] STREAM    LISTENING   1374     /run/systemd/fscck.progress
unix  2      [ ACC ] STREAM    LISTENING   15259    /run/user/1000/gnupg/S.gpg-agent.ssh
unix  2      [ ACC ] STREAM    LISTENING   15261    /run/user/1000/gnupg/S.gpg-agent
unix  2      [ ACC ] STREAM    LISTENING   15263    /run/user/1000/pulse/native
unix  2      [ ACC ] STREAM    LISTENING   15265    /run/user/1000/pipewire-0
unix  2      [ ACC ] STREAM    LISTENING   1380     /run/systemd/journal/stdout
unix  2      [ ACC ] STREAM    LISTENING   15267    /run/user/1000/pipewire-0-manager
unix  2      [ ACC ] STREAM    LISTENING   1382     /run/udev/control
unix  2      [ ACC ] STREAM    LISTENING   20528    /run/user/1000/keyring/pkcs11
unix  2      [ ACC ] STREAM    LISTENING   19574    /run/user/1000/at-spi/bus_0
unix  2      [ ACC ] STREAM    LISTENING   1440     /run/systemd/journal/io.systemd.journal
unix  2      [ ACC ] STREAM    LISTENING   93386    /run/user/1000/vscode-6d67b5db-1.84-main.sock
unix  2      [ ACC ] STREAM    LISTENING   89902    /run/user/1000/vscode-git-0ab30a2c6a.sock
unix  2      [ ACC ] STREAM    LISTENING   14762    /run/dbus/system_bus_socket
unix  2      [ ACC ] STREAM    LISTENING   14764    /run/pcscd/pcscd.comm
unix  2      [ ACC ] STREAM    LISTENING   14766    /run/spice-vdagentd/spice-vdagent-sock
unix  2      [ ACC ] STREAM    LISTENING   14908    @/tmp/.X11-unix/X0
unix  2      [ ACC ] STREAM    LISTENING   19585    @/tmp/.ICE-unix/792

```

Figure 7: Listening TCP ports

```

$ netstat -lt
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State

```

Figure 8: Listening UDP ports

```

$ netstat -lu
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp6      0      0 fe80::e07:dhcpx6-client [::]:*

```

4

Figure 9: All UDP ports

```

$ netstat -au | head
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
udp      0      0 192.168.64.3:bootpc     192.168.64.1:bootps    ESTABLISHED
udp6     0      0 fe80::e07:dhcpx6-client [::]:*

```

Figure 10: All TCP ports

```

$ netstat -at | head
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State

```

3. ifconfig

- Description:
Command to configure network interfaces present in the system.
- Syntax: `ifconfig [ -options ]`

Table 3: Option and their meanings

Option	Meaning
-a -s	All interfaces available, even if they are not running. display a short list output

- Output:

Figure 11: Short form output

```
$ ifconfig -s
```

Iface	MTU	RX-OK	RX-ERR	RX-DRP	RX-OVR	TX-OK	TX-ERR	TX-DRP	TX-OVR	Flg	
eth0	1500	95117	0	0	0	12700	0	0	0	0	BMRU
lo	65536	4	0	0	0	4	0	0	0	0	LRU

5

Figure 12: Output of ifconfig

```
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.64.3 netmask 255.255.255.0 broadcast 192.168.64.255
    inet6 fd2d:525f:37f:2290:e07c:59ff:fe4c:45fa prefixlen 64 scopeid 0<global>
    inet6 fe80::e07c:59ff:fe4c:45fa prefixlen 64 scopeid 0<link>
    inet6 fd2d:525f:37f:2290:6f8d:b884:f636:ab15 prefixlen 64 scopeid 0<global>
    ether e2:7c:59:fc:45:fa txqueuelen 1000 (Ethernet)
    RX packets 95117 bytes 139490886 (133.0 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12700 bytes 1046515 (1021.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 4 bytes 240 (240.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 4 bytes 240 (240.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Figure 13: ifconfig of a particular interface

```
$ ifconfig eth0
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.64.3 netmask 255.255.255.0 broadcast 192.168.64.255
    inet6 fd2d:525f:37f:2290:e07c:59ff:fe4c:45fa prefixlen 64 scopeid 0<global>
    inet6 fe80::e07c:59ff:fe4c:45fa prefixlen 64 scopeid 0<link>
    inet6 fd2d:525f:37f:2290:6f8d:b884:f636:ab15 prefixlen 64 scopeid 0<global>
    ether e2:7c:59:fc:45:fa txqueuelen 1000 (Ethernet)
    RX packets 95117 bytes 139490886 (133.0 MiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 12700 bytes 1046515 (1021.9 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

4. nslookup

- Description:

Nslookup is a program to query Internet domain name servers for finding IP address given a URL or vice-versa.

- Syntax: nslookup [-option] [name | -] [server]
- Output:

Figure 14: nslookup given URL

```
$ nslookup www.google.com
Server: 192.168.64.1
Address: 192.168.64.1#53

Non-authoritative answer:
Name: www.google.com
Address: 142.250.195.228
Name: www.google.com
Address: 2404:6800:4007:822::2004
```

6

5. traceroute

- Description:
This command traces the route that packets takes to reach the host. It will show how many hops it takes to reach the host and time between each hop
- Syntax: traceroute [-option] *hhosti*.
- Output:

Figure 15: traceroute of www.google.com

```
$ traceroute google.com
traceroute to google.com (142.250.182.78), 30 hops max, 60 byte packets
 1 192.168.64.1 (192.168.64.1) 0.445 ms 0.411 ms 0.401 ms
 2 10.106.0.1 (10.106.0.1) 8.907 ms 8.898 ms 8.891 ms
 3 10.106.0.2 (10.106.0.2) 3.573 ms 4.364 ms 4.355 ms
 4 * * *
 5 * * *
 6 * * *
 7 * * *
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
```

6. ping

- Description:
Ping is a computer network administration software utility used to test the reachability of a host on an

Internet Protocol network.

- Syntax: ping [-options] *hdestinationi*
- Output:

Figure 16: ping request www.yahoo.com

```
$ ping google.com
PING google.com (142.250.182.78) 56(84) bytes of data.
64 bytes from maa05s20-in-f14.1e100.net (142.250.182.78): icmp_seq=1 ttl=56 time=6.90 ms
64 bytes from maa05s20-in-f14.1e100.net (142.250.182.78): icmp_seq=2 ttl=56 time=6.21 ms
^C
— google.com ping statistics —
2 packets transmitted, 2 received, 0% packet loss, time 1004ms
rtt min/avg/max/mdev = 6.211/6.556/6.902/0.345 ms
```

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 2
Echo Client

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Echo Client Message Over Network

```
#include <stdio.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char **argv)
{

    int socketID, portNum;
    struct sockaddr_in serverAddr;

    socketID = socket(AF_INET, SOCK_STREAM, 0); // IPv4 TCP/UDP IP

    serverAddr.sin_family = AF_INET;
    portNum = atoi(argv[1]);
    serverAddr.sin_port = htons(portNum);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    //printf("Server : serversoc %d\n", socketID);
```

```

    bind(socketID, (struct sockaddr*) &serverAddr,
sizeof(serverAddr));

listen(socketID,10);
int clientSocket = accept(socketID, NULL, NULL);

//printf("Server : clientsoc %d\n",clientSocket);
char data[255] = " ";
for(int i=2; i< argc; i++)
{

    strcat(data,argv[i]);

    strcat(data," ");
}

send(clientSocket, data, sizeof(data),0);
recv(clientSocket, data, sizeof(data),0);

printf("Echoed Message from client: %s", data);

close(clientSocket);
close(socketID);

return 0; }

```

client.c

```

#include <stdio.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>

```

```
int main(int argc, char **argv)
{

    int socketID, portNum;
    struct sockaddr_in serverAddr;


    socketID = socket(AF_INET,SOCK_STREAM,0); // IPv4 TCP/UDP IP


    serverAddr.sin_family = AF_INET;
    portNum = atoi(argv[1]);
    serverAddr.sin_port = htons(portNum);
    serverAddr.sin_addr.s_addr = INADDR_ANY;


    //printf("Client : %d\n",socketID);


    connect(socketID, (struct sockaddr*) &serverAddr,
sizeof(serverAddr));


    char data[255];
    recv(socketID, data, sizeof(data),0);


    printf("Message from server: %s", data);

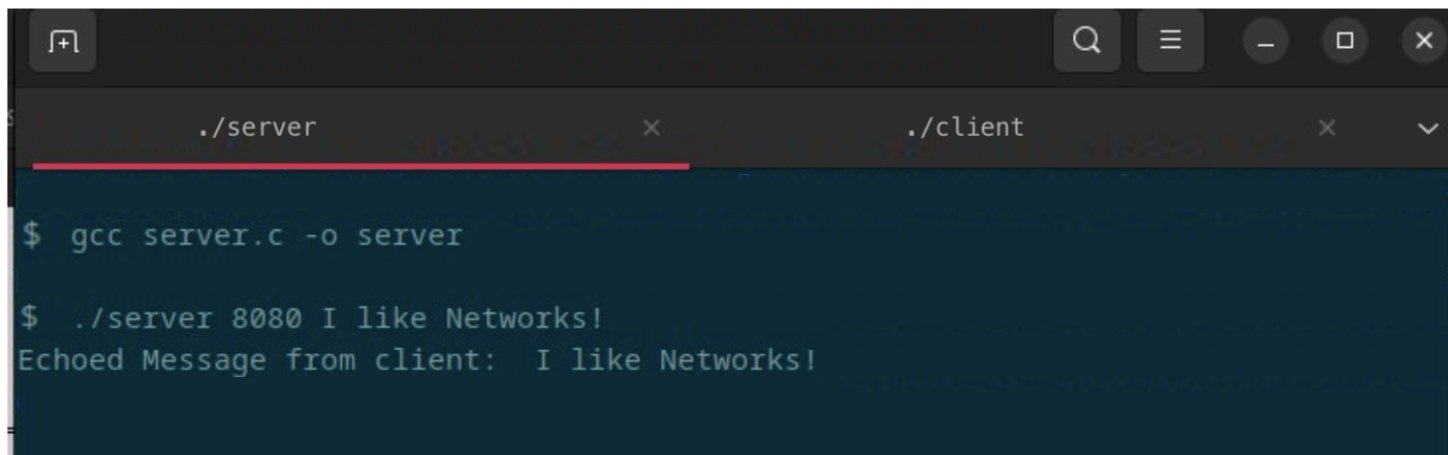

    //sendto(socketID, data, sizeof(data),0,(struct
sockaddr*) &serverAddr, sizeof(serverAddr));


    send(socketID, data, sizeof(data),0);
    close(socketID);
    return 0;

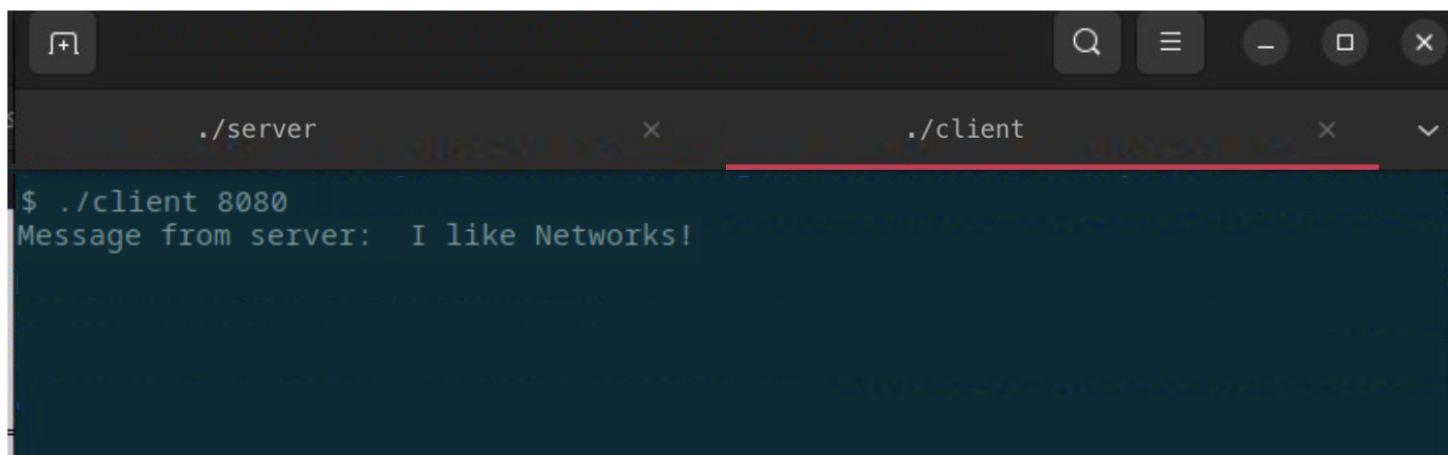
}
```

Output:

O/P:

A terminal window with two tabs: './server' (selected) and './client'. The terminal shows the compilation of server.c and its execution with port 8080 and a message. The output is an echoed message from the client.

```
$ gcc server.c -o server
$ ./server 8080 I like Networks!
Echoed Message from client: I like Networks!
```

A terminal window with two tabs: './server' and './client' (selected). The terminal shows the execution of the client with port 8080, which receives a message from the server.

```
$ ./client 8080
Message from server: I like Networks!
```

Learning Outcomes:

I learned how to echo text using TCP.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 3
File Transfer Using TCP

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

File Transfer In C

server.c

```
#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <unistd.h>
#include <dirent.h>
#include <string.h>
#include <libgen.h>
#include <sys/stat.h>
#include <fcntl.h>
```

```
char filepath[PATH_MAX] = " ";
```

```
void FileTransfer(const char *path, const char *filename) {
    DIR *currentdirectory;
    struct stat s;
    struct dirent *file;

    if ((currentdirectory = opendir(path)) == NULL) {
        perror("opendir");
        return;
    }
}
```

```
};
```

```
while ((file = readdir(currentdirectory)) != NULL) {  
    char fullpath[PATH_MAX]; // Use a large enough buffer  
    strcpy(fullpath, path);
```

```
    strcat(fullpath, "/");  
    strcat(fullpath, file->d_name);
```

```
    stat(fullpath, &s);  
    if (!strcmp(file->d_name, filename)) {
```

```
        printf("Found: %s\n", fullpath);  
        strcpy(filepath, fullpath);  
        break;
```

```
}
```

```
if (S_ISDIR(s.st_mode)) {  
    if (strcmp(file->d_name, ".") != 0 && strcmp(file-
```

```
>d_name, ".") != 0) {  
        //printf("\n%s", fullpath);
```

```
call  
}
```

```
}}
```

```
FileTransfer(fullpath, filename); // Recursive
```

```
    closedir(currentdirectory);  
}
```

```
char* FTP(char* filepath)  
{
```

```
    static char buffer[1024];  
    if(!strcmp(filepath, ""))
```

```
    {  
        strcpy(buffer, "FileNotFound");
```

```
} else
```

```
{  
    int fd = open(filepath, O_RDONLY);  
    struct stat s;  
    stat(filepath, &s);
```

```
    read(fd, buffer, s.st_size);
```

```
}
```

```
    return buffer;

}

int main(int argc, char** argv)
{

    int socket_id, client_id;
    struct sockaddr_in server, client;
    char filename[20];
    char* data;

    if(argc<2) {

        printf("Usage: %s <port>\n",argv[0]);
        exit(1);

    }

    socket_id = socket(AF_INET,SOCK_STREAM,0);

    server.sin_family = AF_INET;
    server.sin_port = htons(atoi(argv[1]));
    server.sin_addr.s_addr = INADDR_ANY;


    bind(socket_id, (struct sockaddr *) &server, sizeof(server));
    printf("Binding...\n");

    listen(socket_id, 3);
    printf("listening...\n");
```

```
socklen_t size = sizeof(client);
```

```
client_id = accept(socket_id,(struct sockaddr*) &client,  
&size);
```

```
if(client_id < 0)  
{
```

```
    perror("Connection Failed\n");  
}
```

```
printf("Connected...\n");
```

```
int bytes_read = recv(client_id,filename,sizeof(filename),0);  
filename[bytes_read] = '\0';
```

```
FileTransfer("/home/",filename);  
data = FTP(filepath);
```

```
send(client_id,data,strlen(data),0);
```

```
close(client_id);  
close(socket_id);
```

```
return 0; } client.c
```

```
#include <stdio.h>  
#include <sys/socket.h>  
#include <netinet/in.h>  
#include <stdlib.h>  
#include <unistd.h>  
#include <string.h>
```

```
int main(int argc, char** argv)
{

    int socket_id;
    struct sockaddr_in client;

    char data[1024];

    if(argc<3) {

        printf("Usage: %s <port> <filename>\n",argv[0]);
        exit(1);

    }
    socket_id = socket(AF_INET,SOCK_STREAM,0);

    client.sin_family = AF_INET;
    client.sin_port = htons(atoi(argv[1]));
    client.sin_addr.s_addr = INADDR_ANY;

    connect(socket_id, (struct sockaddr*) &client,
sizeof(client));

    printf("Connected to server!\n");
    send(socket_id, argv[2], strlen(argv[2]),0);
    recv(socket_id, data, sizeof(data),0);
    printf("Data: %s",data);
    close(socket_id);

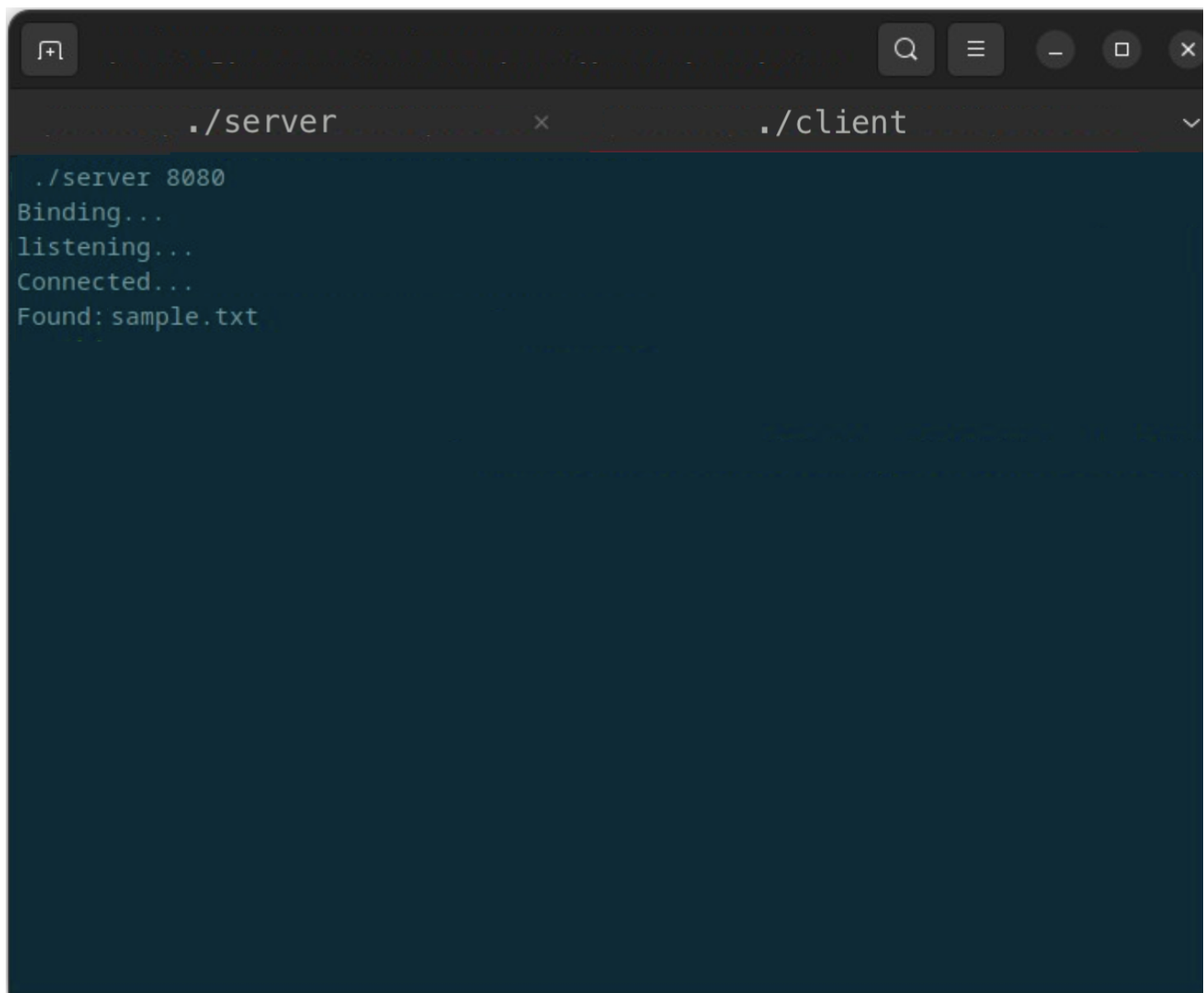
    return 0;
```

```
}
```

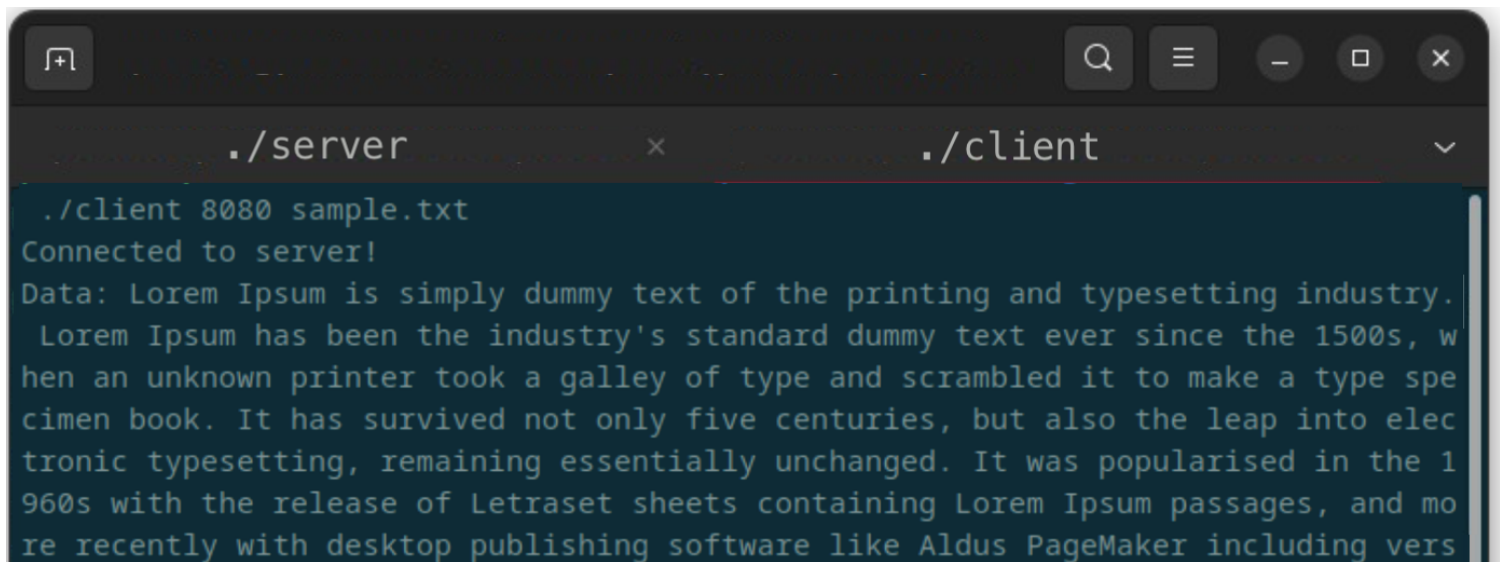
sample.txt

Lorem Ipsum is simply dummy text of the printing and typesetting industry. Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, when an unknown printer took a galley of type and scrambled it to make a type specimen book. It has survived not only five centuries, but also the leap into electronic typesetting, remaining essentially unchanged. It was popularised in the 1960s with the release of Letraset sheets containing Lorem Ipsum passages, and more recently with desktop publishing software like Aldus PageMaker including versions of Lorem Ipsum.

Output:



O/P:

A terminal window with a dark background and light green text. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a search bar. Below the title bar, there are two tabs: './server' and './client'. The './client' tab is active. The terminal content shows the command './client 8080 sample.txt' being executed, followed by the output 'Connected to server!' and a large block of Lorem Ipsum text.

```
./client 8080 sample.txt
Connected to server!
Data: Lorem Ipsum is simply dummy text of the printing and typesetting industry.
Lorem Ipsum has been the industry's standard dummy text ever since the 1500s, w
hen an unknown printer took a galley of type and scrambled it to make a type spe
cimen book. It has survived not only five centuries, but also the leap into elec
tronic typesetting, remaining essentially unchanged. It was popularised in the 1
960s with the release of Letraset sheets containing Lorem Ipsum passages, and mo
re recently with desktop publishing software like Aldus PageMaker including vers
```

Learning Outcomes:

I learned how to transfer files using socket programming.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 6
MultiUserChat

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Chat in C.

server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <pthread.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
```

```
int clientCount = 0;
```

```
static pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
static pthread_cond_t cond = PTHREAD_COND_INITIALIZER;
```

```
struct client{
```

```
    int index;
    int sockID;
    struct sockaddr_in clientAddr;
    int len;
```

```
};
```

```
struct client Client[1024];  
pthread_t thread[1024];
```

```
void * doNetworking(void * ClientDetail){
```

```
    struct client* clientDetail = (struct client*)  
    ClientDetail;
```

```
    int index = clientDetail -> index;  
    int clientSocket = clientDetail -> sockID;
```

```
    printf("Client %d connected.\n",index + 1);  
    while(1){
```

```
        char data[1024];  
        int read = recv(clientSocket,data,1024,0);  
        data[read] = '\0';
```

```
        char output[1024];  
        if(strcmp(data,"LIST") == 0){
```

```
            int l = 0;  
            for(int i = 0 ; i < clientCount ; i ++){
```

```
                if(i != index)  
                    l += snprintf(output +
```

```
l,1024,"Client %d is at socket %d.\n",i + 1,Client[i].sockID);  
            }
```

```
            send(clientSocket,output,1024,0);  
            continue;
```

```
    }
```

```
if(strcmp(data,"SEND") == 0){
```

```
    read = recv(clientSocket,data,1024,0);  
    data[read] = '\0';
```

```
int id = atoi(data) - 1;
```

```
read = recv(clientSocket,data,1024,0);  
data[read] = '\0';
```

```
    send(Client[id].sockID,data,1024,0);  
}
```

```
}
```

```
return NULL;  
}
```

```
int main(){  
    int serverSocket = socket(PF_INET, SOCK_STREAM, 0);  
    struct sockaddr_in serverAddr;
```

```
    serverAddr.sin_family = AF_INET;  
    serverAddr.sin_port = htons(8080);  
    serverAddr.sin_addr.s_addr = htons(INADDR_ANY);
```

```
    if(bind(serverSocket,(struct sockaddr *) &serverAddr ,
```

```

sizeof(serverAddr)) == -1) return 0;

    if(listen(serverSocket,1024) == -1) return 0;

    printf("Server started listenting on port
8080 .....\\n");

while(1){

    Client[clientCount].sockID = accept(serverSocket,
(struct sockaddr*) &Client[clientCount].clientAddr,
&Client[clientCount].len);

    Client[clientCount].index = clientCount;

    pthread_create(&thread[clientCount], NULL,
doNetworking, (void *) &Client[clientCount]);

    clientCount ++;
}

for(int i = 0 ; i < clientCount ; i ++)
    pthread_join(thread[i],NULL);

}

```

client.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

```

```
#include <fcntl.h>
#include <pthread.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <arpa/inet.h>
```

```
void * doRecieving(void * sockID){
    int clientSocket = *((int *) sockID);
    while(1){
```

```
        char data[1024];
```

```
        int read = recv(clientSocket,data,1024,0);
        data[read] = '\0';
        printf("%s\n",data);
```

```
    } }
```

```
int main(){
    int clientSocket = socket(PF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serverAddr;
```

```
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(8080);
    serverAddr.sin_addr.s_addr = htonl(INADDR_ANY);
```

```
    if(connect(clientSocket, (struct sockaddr*) &serverAddr,
sizeof(serverAddr)) == -1) return 0;
```

```
printf("Connection established .....\\n");
```

```
pthread_t thread;
```

```
pthread_create(&thread, NULL, doReceiving, (void *)  
&clientSocket );
```

```
while(1){
```

```
    char input[1024];  
    scanf("%s",input);
```

```
    if(strcmp(input,"LIST") == 0){  
        send(clientSocket,input,1024,0);
```

```
}
```

```
if(strcmp(input,"SEND") == 0){  
    send(clientSocket,input,1024,0);
```

```
    scanf("%s",input);  
    send(clientSocket,input,1024,0);
```

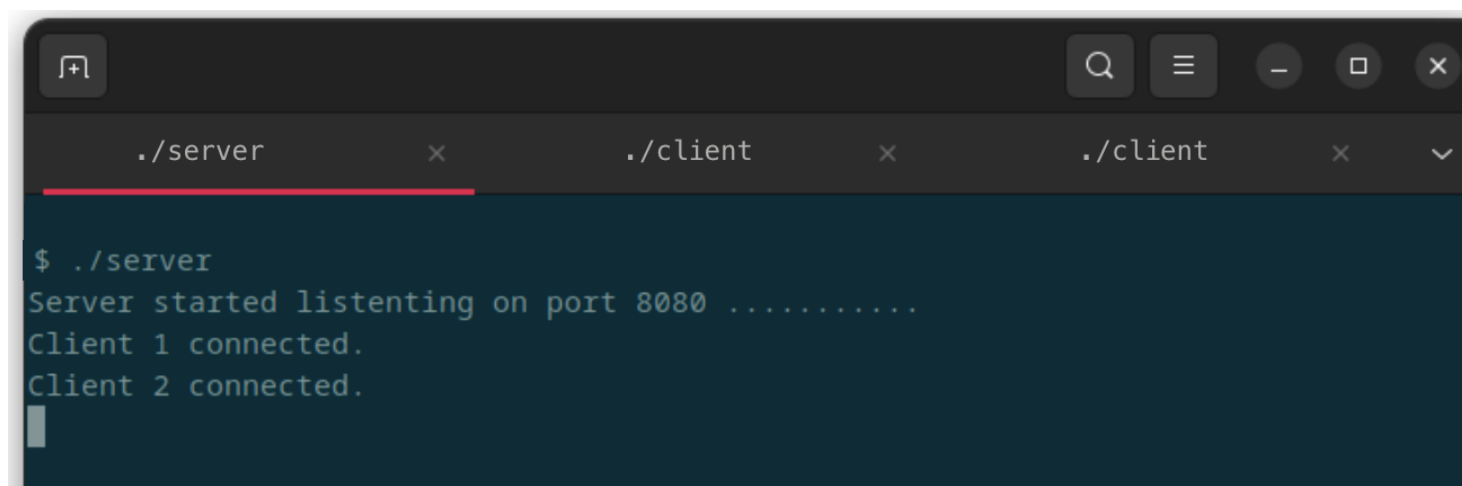
```
    scanf("%[^\n]s",input);
```

```
    send(clientSocket,input,1024,0);
```

```
}
```

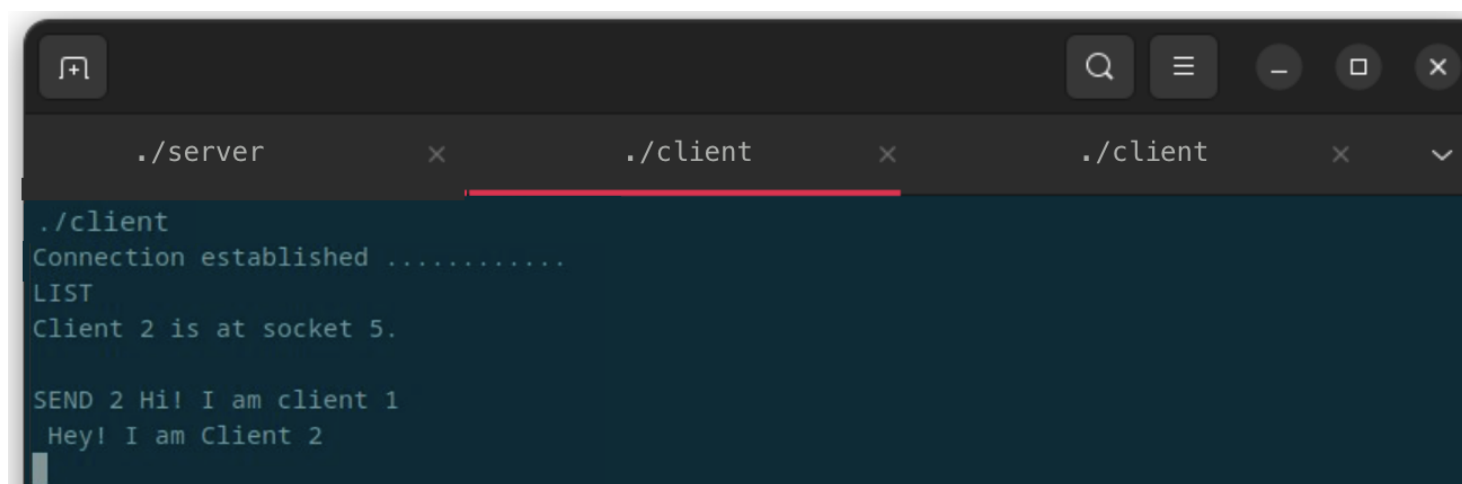
```
}}
```

O/P:



A terminal window with three tabs: `./server`, `./client`, and `./client`. The `./server` tab is active and shows the following output:

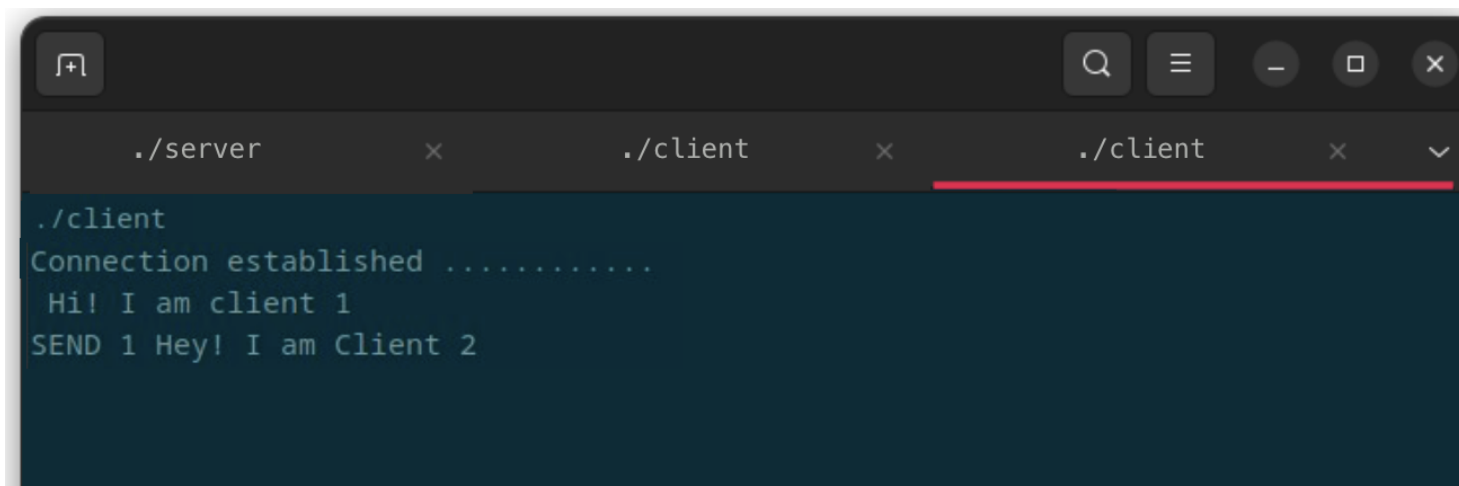
```
$ ./server
Server started listenting on port 8080 .....
Client 1 connected.
Client 2 connected.
```



A terminal window with three tabs: `./server`, `./client`, and `./client`. The first `./client` tab is active and shows the following output:

```
./client
Connection established .....
LIST
Client 2 is at socket 5.

SEND 2 Hi! I am client 1
Hey! I am Client 2
```

A terminal window with a dark background and light-colored text. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a search icon. Below the title bar, there are three tabs: './server', './client', and './client'. The third tab is selected and highlighted with a red underline. The terminal content shows the following text:

```
./client
Connection established .....
Hi! I am client 1
SEND 1 Hey! I am Client 2
```

Learning Outcomes:

I learned how to simulate ARP using socket programming.

Ex: 9**ADDRESS RESOLUTION PROTOCOL (ARP)**

Simulate ARP using socket programming.

Server should perform the following:

1. Consider the server as a host or a router.
2. Enter hosts/routers' IP address and MAC address.
3. Listen for any number of client (for broadcasting purpose)
4. Enter the packet details received from a host or its own packet to sent to a destination.

The details are:

- i. Source IP address
 - ii. Source MAC address
 - iii. Destination IP address
 - iv. 16 bit data
5. Develop an ARP Request packet which is to be broadcasted to all clients. Query packet should contain

SourceIP | SourceMAC | DestinationIP

6. When an ARP Reply is received with the Destination MAC address, send the packet to the corresponding destination with format

SourceIP | SourceMAC | DestinationIP | Destination MAC Address | 16 bit data

7. Also check the validity of IP and MAC address.

Client should do the following:

1. Can have any number of clients(depends on the backlog). A single program is enough for any number of clients.
2. Enter the clients own IP and MAC.
3. When an ARP Request packet is received check whether the Destination IP is its own IP.
4. If not no reply.
5. If yes respond with ARP Reply packet.

SourceIP | SourceMAC | DestinationIP | DestinationMAC

6. Then receive the packet from the server and display it.

Sample Input and Output

Server

Enter the details of packet received.

Destination IP : 155.157.65.128

Source IP : 123.128.34.56

Source MAC : AF-45-E5-00-97-12

16 bit data : 1011110000101010

Developing ARP Request packet

123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

The ARP Request packet is broadcasted.

Waiting for ARP Reply...

ARP Reply received 123.128.34.56 | AF-45-E5-00-97-12 |
155.157.65.128 | 45-DA-62-21-1A-B2

Sending the packet to : 45-DA-62-21-1A-B2

Packet Sent: 123.128.34.56 | AF-45-E5-00-97-12 |
155.157.65.128 | 45-DA-62-21-1A-B2 | 011110000101010

Client 1

Enter the IP address : 165.43.158.158

Enter the Mac address : 09-DF-90-26-6C-09

ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

IP address does not match.

Client 2

Enter the IP address : 155.157.65.128

Enter the Mac address : 45-DA-62-21-1A-B2

ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128

IP address matches

ARP Reply Sent : 123.128.34.56 | AF-45-E5-00-97-12 |

Received Packet is : 155.157.65.128 | 45-DA-62-21-1A-B2 |
: 123.128.34.56 | AF-45-E5-00-97-12 |
155.157.65.128 | 45-DA-62-21-1A-B2 | 011110000101010

Client 3

Enter the IP address : 15.143.158.18
Enter the Mac address : 19-0F-01-63-C7-D4
ARP Request Received : 123.128.34.56 | AF-45-E5-00-97-12 | 155.157.65.128
IP address does not match.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 10
DNS Resolution using UDP

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

DNS Resolution using UDP

server.c

Aim:

` Simulation of DNS in socket programming in C

server.c

```
// server program for udp connection
#include <stdio.h>
#include <strings.h>
#include <stdlib.h>
```

```
#include <sys/types.h>
```

```

#include <arpa/inet.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>

#define PORT 5000
#define MAXLINE 1000
#define MAX 10

typedef struct
{

    char * domain_name;

    char * IP;
} dom;

dom domain_arr[MAX];

// Driver code
int main()
{

    char buffer[100];
    char *message = "Server connected";
    int listenfd, len;
    struct sockaddr_in servaddr, cliaddr;
    bzero(&servaddr, sizeof(servaddr));

    // Create a UDP Socket
    listenfd = socket(AF_INET, SOCK_DGRAM, 0);
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port = htons(PORT);
    servaddr.sin_family = AF_INET;

    //reqd ip
    char *correct = (char*)malloc(MAXLINE);

```

```

char *modify = (char*)malloc(MAXLINE);
char *new_ip = (char*)malloc(MAXLINE);
strcpy(correct,"IP not found");
//create table
domain_arr[0].domain_name = "youtube.com";


domain_arr[0].IP = "1.2.3.4";
domain_arr[1].domain_name = "google.com";
domain_arr[1].IP = "6.7.8.9";
domain_arr[2].domain_name = "chatgpt.ai";
domain_arr[2].IP = "10.11.12.13";


for(int i=0;i<3;i++)
{

printf("IP %s Domain name = %s \n",domain_arr[i].IP,
domain_arr[i].domain_name);


}
// bind server address to socket descriptor
bind(listenfd, (struct sockaddr*)&servaddr,

sizeof(servaddr));


//receive the datagram
len = sizeof(cliaddr);
int n = recvfrom(listenfd, buffer, sizeof(buffer),

0, (struct sockaddr*)&cliaddr,&len); //receive
message from client


printf("Buffer is %s ",buffer);
/*
int n1 = recvfrom(listenfd, modify, sizeof(buffer),

```

```
0, (struct sockaddr*)&cliaddr,&len);  
printf("Modify = ? %s \n ",modify);*/
```

```
printf("Want to modify? yes or no ");  
fgets(modify,MAXLINE,stdin);  
printf("The value is %d ",strncmp(modify,"yes",3));  
if(strncmp(modify,"yes",3)==0)  
{
```

```
printf("Enter IP to replace. Will replace if found
```

```
");
```

```
    //printf("Result %d \n",strncmp(domain_arr[i].domain_name,buffer,sizeof(domain_arr[0  
]));
```

```
if(strncmp(domain_arr[i].domain_name,buffer,sizeof(domain_arr[0  
]))+10 == 0)
```

```
{  
    printf("Found the Required IP  %s  Domain name
```

```
= %s ",domain_arr[i].IP, domain_arr[i].domain_name);  
    if(strncmp(modify,"yes",3)==0)
```

```
{  
    domain_arr[i].IP = new_ip;
```

```
}  
strcpy(correct,domain_arr[i].IP);
```

```
}}
```

```
buffer[n] = '\0';  
puts(buffer);
```

```
// send the response  
sendto(listenfd, correct, MAXLINE, 0,
```

```
(struct sockaddr*)&cliaddr, sizeof(cliaddr));
```

```
if(strncmp(modify,"yes",3)==0)  
{
```

```
    sendto(listenfd, new_ip, MAXLINE, 0,
```

```
    (struct sockaddr*)&cliaddr, sizeof(cliaddr));  
}
```

```
for(int i=0;i<3;i++)  
{
```

```
    printf("IP  %s  Domain name = %s \n",domain_arr[i].IP,  
domain_arr[i].domain_name);
```

```
}}
```

```
//client.c
```

```
// udp client driver program
```

```
#include <stdio.h>
```

```
#include <strings.h>
```

```
#include <string.h>
```

```
#include <sys/types.h>
```

```
#include <arpa/inet.h>
```

```
#include <sys/socket.h>
```

```
#include<netinet/in.h>
```

```
#include<unistd.h>
```

```
#include<stdlib.h>
```

```
#define PORT 5000
```

```
#define MAXLINE 100
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    char buffer[100];
```

```
    char *message = (char *)malloc(MAXLINE);
```

```
    printf("Enter domain name : ");
```

```
    fgets(message,MAXLINE,stdin);
```

```
    int sockfd, n;
```

```
    struct sockaddr_in servaddr;
```

```
// clear servaddr
```

```
    bzero(&servaddr, sizeof(servaddr));
```

```
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
```

```

servaddr.sin_port = htons(PORT);
servaddr.sin_family = AF_INET;


// create datagram socket
sockfd = socket(AF_INET, SOCK_DGRAM, 0);


// connect to server


if(connect(sockfd, (struct sockaddr *)&servaddr,
sizeof(servaddr)) < 0)

{
    printf("\n Error : Connect Failed \n");
    exit(0);
}


// request to send datagram
// no need to specify server address in sendto
// connect stores the peers IP and port


sendto(sockfd, message, MAXLINE, 0, (struct
sockaddr*)NULL, sizeof(servaddr));


/*printf("Do you want to update the ip? yes or no: ");
char *x = (char *)malloc(MAXLINE);
fgets(x,MAXLINE,stdin);
printf("\n Sending %s ",x );


sendto(sockfd, x, MAXLINE, 0, (struct sockaddr*)NULL,
sizeof(servaddr));


if(strncmp(x,"yes",3)!=0)
{
    char *x1 = (char *)malloc(MAXLINE);
    printf("Enter the new IP. Will update it if found");
    fgets(x1,MAXLINE,stdin);
    sendto(sockfd, x1, MAXLINE, 0, (struct sockaddr*)NULL,

```

```
sizeof(servaddr));
    }*/

    // waiting for response

    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct
sockaddr*)NULL, NULL);

    printf("The IP of the domain is %s \n" ,buffer);

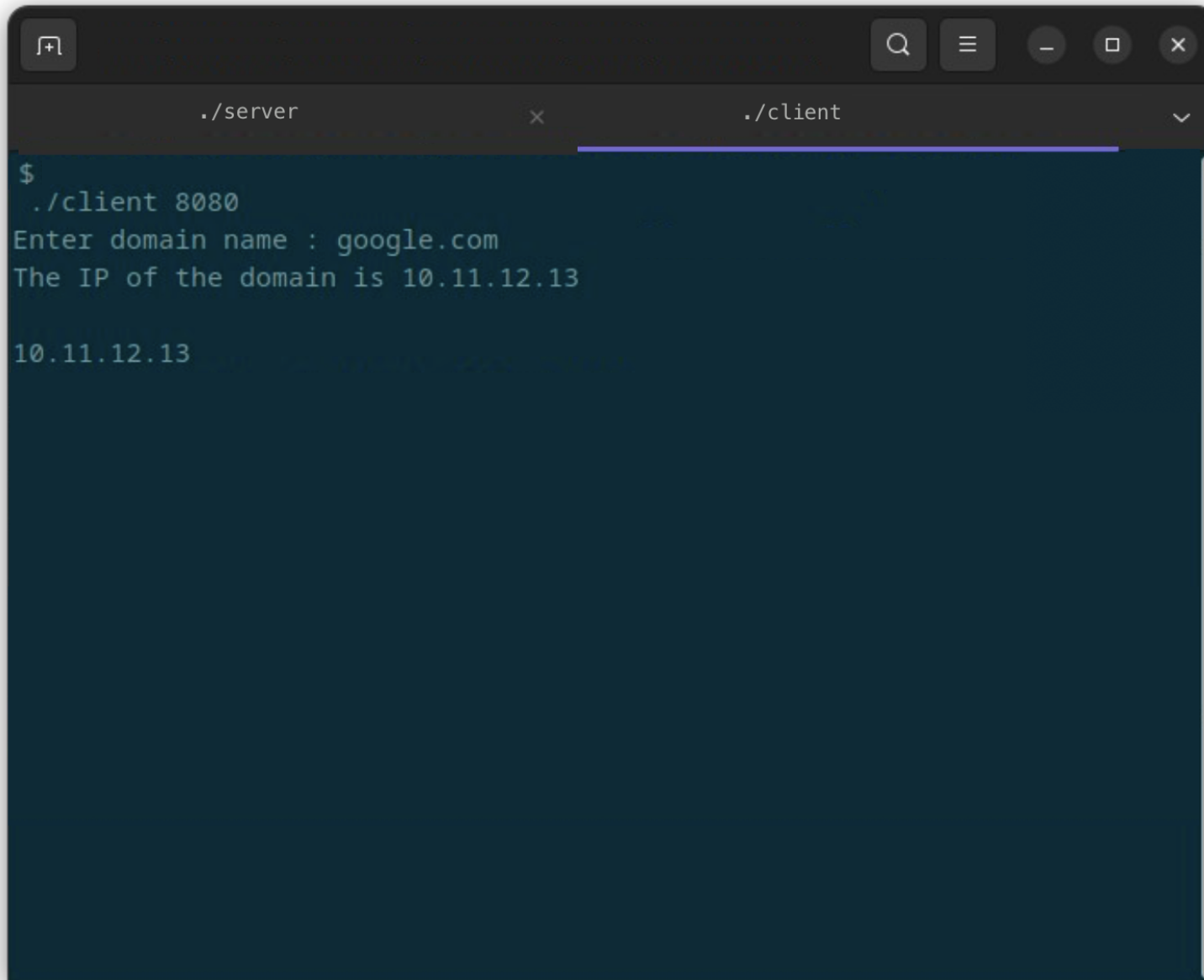
    recvfrom(sockfd, buffer, sizeof(buffer), 0, (struct
sockaddr*)NULL, NULL);

    printf("%s" ,buffer);

    // close the descriptor

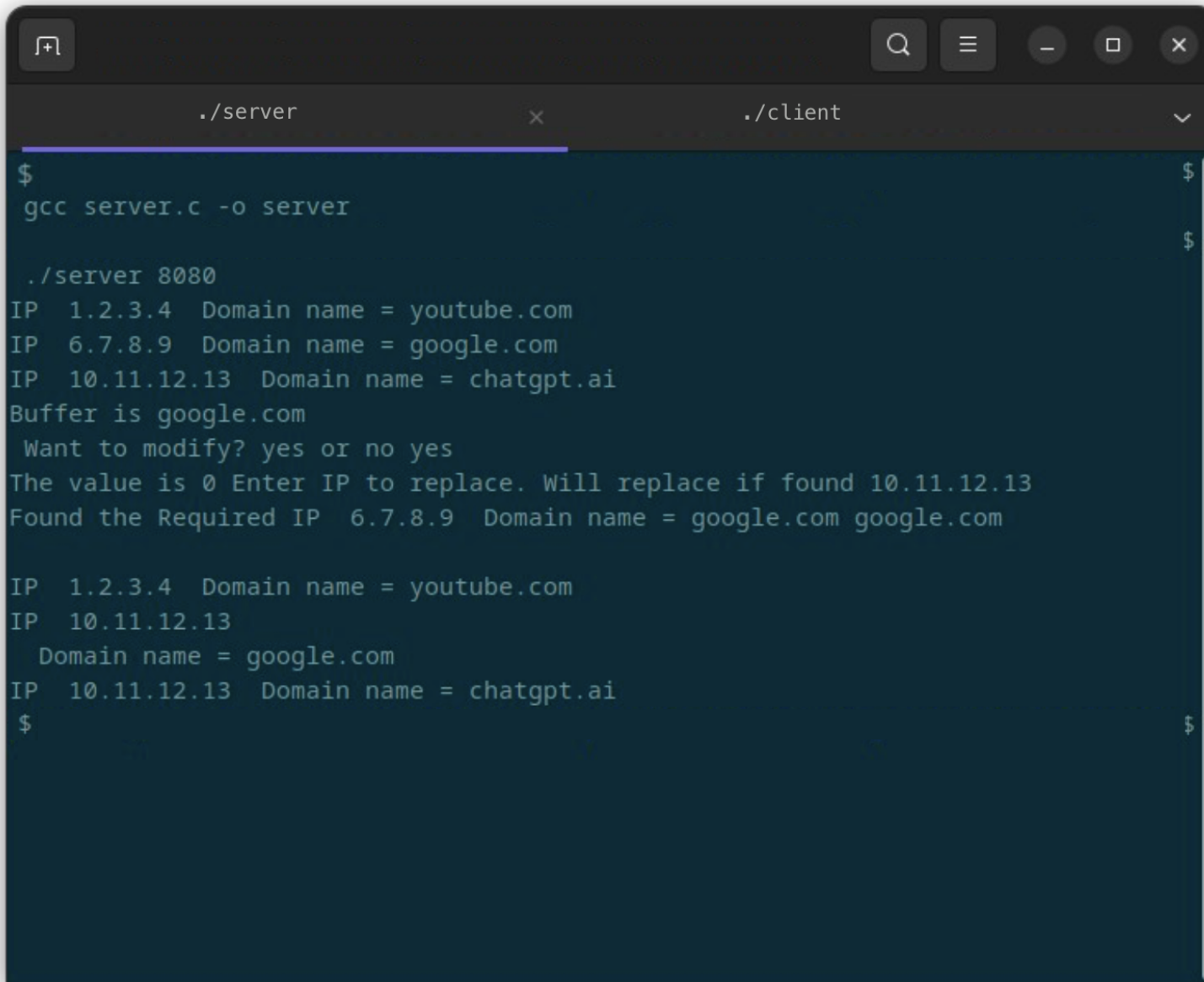
    close(sockfd);
}
```

O/P:



A terminal window with a dark background and light green text. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a search icon. Below the title bar, there are two tabs: `./server` and `./client`, with `./client` being the active tab. The terminal content shows a shell prompt `$`, followed by the command `./client 8080`. The program then prompts for a domain name, which is entered as `google.com`. The program outputs `The IP of the domain is 10.11.12.13`. Finally, the IP address `10.11.12.13` is printed on a new line.

```
$  
./client 8080  
Enter domain name : google.com  
The IP of the domain is 10.11.12.13  
  
10.11.12.13
```

A terminal window with two tabs: './server' and './client'. The './server' tab is active and shows the following commands and output:

```
$  
gcc server.c -o server  
  
./server 8080  
IP 1.2.3.4 Domain name = youtube.com  
IP 6.7.8.9 Domain name = google.com  
IP 10.11.12.13 Domain name = chatgpt.ai  
Buffer is google.com  
Want to modify? yes or no yes  
The value is 0 Enter IP to replace. Will replace if found 10.11.12.13  
Found the Required IP 6.7.8.9 Domain name = google.com google.com  
  
IP 1.2.3.4 Domain name = youtube.com  
IP 10.11.12.13  
Domain name = google.com  
IP 10.11.12.13 Domain name = chatgpt.ai  
$
```

Learning Outcomes:

I learned how to implement flow control using socket programming.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 10
Flow Control

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Flow Control (Go-Back N) socket programming in C

server.c

```
#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/in.h>
#include <ctype.h>
#include <netdb.h>
#define MAX_PENDING 5
#define MAX_LINE 256

int main(int argc, char **argv){
    struct sockaddr_in sin;
    int pos,len,s,new_s,ack=0;

    char transmit[4] = "no";
    char buffer[MAX_LINE],ch;
    //char sentback[50];
    bzero((char *)&sin, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_addr.s_addr = INADDR_ANY;
    sin.sin_port = htons(atoi(argv[1]));

    /* setup passive open */
    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {
```

```
perror("simplex-talk: socket");
```

```
exit(1); }
```

```
if ((bind(s, (struct sockaddr *)&sin, sizeof(sin))) < 0)  
{
```

```
perror("simplex-talk: bind");
```

```
exit(1); }
```

```
listen(s, MAX_PENDING);
```

```
if ((new_s = accept(s, (struct sockaddr *)&sin, &len)) <  
0) {
```

```
perror("simplex-talk: accept");
```

```
exit(1); }
```

```
printf("Established connection with client on port %d\n",  
atoi(argv[1]));
```

```
while (((len=recv(new_s,buffer,sizeof(buffer),0))>0)){  
    printf("Received: %s\n",buffer);  
    pos = buffer[4] - '0';  
    printf("pos:%d\n",pos);
```

```

        if(pos == ack){
            ack++;

        }
        printf("Sent ACK%d\n",ack);
        send(new_s,&ack,sizeof(int),0);

    }
    close(new_s);
    close(s);

}

```

//client.c

```

#include <stdio.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <netinet/in.h>
#include <ctype.h>
#include <netdb.h>
#define MAX_PENDING 5

```

```

#define MAX_LINE 256

```

```

void packet_maker(char *str,char **packets,int rem,int q){
    char laststr[5]="";
    char packet_string[5]="";
    char zero = '0';

```

```

    int len,ind=0;
    len = strlen(str);
    //printf("%d\n",rem);
    int start = 0;
    for(int i =0;i<q;i++){

```

```

strcpy(packet_string, "");
for(int j=0;j<4;j++){

    strncat(packet_string,&str[start++],1);
}

strcpy(packets[ind++],packet_string);

//printf("\n");
}

if(rem!=0){
    int pos = 4 - rem;
    strcpy(laststr, "");
    for(int k=0;k<4;k++){

        if(k<pos){
            strncat(laststr,&zero,1);
        } else{

            strncat(laststr,&str[start++],1);
        }

    }

    strcpy(packets[ind++],laststr);
}

void retransmit(char **packets,int socketid,int from,int total)
{

    printf("Retransmitting!\n");
    for(int a=from;a<total;a++){

        send(socketid,packets[a],25,0);

    }

    printf("Sent packet %d\n",a);
}

```

```

    }

}

int main(int argc, char **argv) {

    char str[15],dst[2],src[2];
    char string[15]="";
    char msg[4]="yes";
    char transmit[4];

    char **packets;
    int total,length,rem,q,len,s,opt,ack=0,no;
    struct sockaddr_in sin;

    bzero((char *)&sin, sizeof(sin));
    sin.sin_family = AF_INET;
    sin.sin_port = htons(atoi(argv[1]));
    sin.sin_addr.s_addr = inet_addr(argv[2]);
    if ((s = socket(PF_INET, SOCK_STREAM, 0)) < 0) {

        perror("simplex-talk: socket");

    exit(1); }

    if (connect(s, (struct sockaddr *)&sin, sizeof(sin)) < 0)
    {

        perror("simplex-talk: connect");
        close(s);
        exit(1);

    }
    printf("Established connection with server on port %d\n",atoi(argv[1]));

    strcpy(str,"");
    printf("Enter the message: ");
    scanf("%s",str);
    length = strlen(str);

```

```
printf("%d\n",length);
rem = length%4;
q=length/4;
if(rem>0){
```

```
total=q+1;
```

```
}else{
```

```
    total=q;
```

```
}
```

```
packets = (char **)malloc(total* sizeof(char *));
```

```
for (int i = 0; i < total; i++) {
```

```
    packets[i] = (char *)malloc(25 * sizeof(char));
```

```
}
```

```
packet_maker(str,packets,rem,q);
```

```
printf("Total Packets are %d\n",total);
```

```
printf("Enter the source: ");
```

```
scanf("%s",src);
```

```
printf("Enter the destination: ");
```

```
scanf("%s",dst);
```

```
for(int l=0;l<total;l++){
```

```
    strcpy(string,"");
```

```
    snprintf(string,sizeof(string),"F-%s-%d-%s-FCS-  
F",dst,l,packets[l]);
```

```
    strcpy(packets[l],string);  
}
```

```
for(int a=0;a<total;a++){  
    printf("Want to send packet %d[1=yes][0=no]: ",a);  
    scanf("%d",&opt);  
    if(opt==1){
```

```
        send(s,packets[a],25,0);  
        printf("Sent packet %d\n",a);  
        ack = packets[a][4] - '0';  
        recv(s,&no,sizeof(int),0);  
        printf("Received ACK%d\n",no);
```

```
}}
```

```
//printf("ack:%d no:%d\n",ack,no);  
if(ack+1!=no){
```

```
    retransmit(packets,s,no,total);
```

```
}  
  
}
```

O/P:

```
vboxuser@vm-vm: ~/Downloads  
vboxuser@vm-vm:~/Downloads$ gcc flow_serv.c -o flow_serv  
vboxuser@vm-vm:~/Downloads$ ./flow_serv 8080  
Established connection with client on port 8080  
Received: F-B-0-1100-FCS  
pos:0  
Sent ACK1  
Received: F-B-1-1100-FCS  
pos:1  
Sent ACK2  
Received: F-B-3-0000-FCS  
pos:3  
Sent ACK2  
Received: F-B-4-0000-FCS  
pos:4  
Sent ACK2  
Received: F-B-5-0011-FCS  
pos:5  
Sent ACK2  
Received: F-B-2-1110-FCS  
pos:2  
Sent ACK3  
vboxuser@vm-vm:~/Downloads$
```

```
vboxuser@vm-vm: ~/Downloads
vboxuser@vm-vm:~/Downloads$ gcc flow_cl.c -o flow_cl
flow_cl.c: In function 'main':
flow_cl.c:61:31: warning: implicit declaration of function 'inet_addr' [-Wimplicit-function-declaration]
   61 |         sin.sin_addr.s_addr = inet_addr(argv[2]);
      |                               ^
vboxuser@vm-vm:~/Downloads$ ./flow_cl 8080 10.0.2.15
Established connection with server on port 8080
Enter the message: 110011001110000000000011
24
Total Packets are 6
Enter the source: A
Enter the destination: B
Want to send packet 0[1-yes][0-no]: 1
Sent packet 0
Received ACK1
Want to send packet 1[1-yes][0-no]: 1
Sent packet 1
Received ACK2
Want to send packet 2[1-yes][0-no]: 0
Want to send packet 3[1-yes][0-no]: 1
Sent packet 3
Received ACK2
Want to send packet 4[1-yes][0-no]: 1
Sent packet 4
Received ACK2
Want to send packet 5[1-yes][0-no]: 1
Sent packet 5
Received ACK2
Retransmitting!
Sent packet 2
Sent packet 3
Sent packet 4
Sent packet 5
vboxuser@vm-vm:~/Downloads$
```

Learning Outcomes:

I learned how to implement flow control using socket programming.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 11
Error Control

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Error Correction Mechanism via Hamming Code in C

O/P:

Program

1. Server Side

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <stdbool.h>
#include <math.h>
#define MAXLINE 1024
#include "Hamming.h"

int main(int argc, char ** argv)
{
    2
    srand(time(NULL));
    if (argc < 2){
        fprintf(stderr, "Enter port number as argument!\n");
        exit(EXIT_FAILURE);
    }
```

```

int PORT = atoi(argv[1]);
int sockfd, newfd, n, arr[30], count = 0, bin;
char buff[MAXLINE], buffer[MAXLINE], data_t[40];
int i, j, r, total, nob, rem, dig, pos;
long data;
struct sockaddr_in servaddr, cliaddr;
if((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0)
{
    perror("Socket creation failed!");
    exit(1); }
bzero(&servaddr, sizeof(servaddr)); servaddr.sin_family = AF_INET; // IPv4
servaddr.sin_addr.s_addr = INADDR_ANY; servaddr.sin_port = htons(PORT);
if(bind(sockfd, (const struct sockaddr *)&servaddr, sizeof(servaddr)) < 0)
{
    perror("Bind failed!");
    exit(1); }
int len, m;
listen(sockfd, 2);
printf("Enter the data to send: ");
scanf("%lu", &data);
n = countbits(data);
r = log2(n);
r = floor(r);
// Finding number of redundant bits
while(pow(2, r) < n + r + 1)
    r += 1;
printf("\nNo. of redundant bits required: %d\n", r);
total = n + r;
nob = floor(log2(total));
for(i = 1; i <= total; i++)
{
    dig = data % 10;
    if(isapower2(i) == 0)
    {
        arr[total - i] = dig;
        data /= 10; }
    else
        arr[total - i] = 0;
    for(i = 0; i < r; i++)
    }
}
3

{
    for(j = 1; j <= total; j++)
    {
        if((int)(pow(2, i)) != j)
        {

```

```

}}
}
2. Client Side
#include <stdio.h> #include <sys/types.h> #include <sys/socket.h> #include
<netinet/in.h> #include <string.h> #include <unistd.h> #include <arpa/inet.h>
#include <stdlib.h> #include <math.h> #define MAXLINE 1024
bin = binary(j);
if(ispresent(bin, i + 1))
    count += arr[total - j];
    if(count % 2 == 0)
        arr[total - (int)(pow(2, i))] = 0;
else
    arr[total - (int)(pow(2, i))] = 1;
    count = 0;
}
printf("\nData with redundant bits: ");
for(i = 0; i < total; i++)
    printf("%d", arr[i]);
// printf("\nEnter error position: "); // scanf("%d", &pos);
pos = rand() % total + 1;
printf("\nIntroducing error automatically at bit: %d\n", pos);
if(arr[total - pos] == 0)
    arr[total - pos] = 1;
else
    arr[total - pos] = 0;
int k = 0;
long num = 0;
for(i = total - 1; i >= 0; i--)
{
    num += pow(10, k) * arr[i];
    k++; }
sprintf(data_t, "%lu", num);
printf("Data transmitted is %s\n", data_t);
len = sizeof(cliaddr);
newfd = accept(sockfd, (struct sockaddr*)&cliaddr, &len);
m = write(newfd, data_t, sizeof(data_t));
#include "Hamming.h"
4

int main(int argc, char **argv)
{
    if (argc < 2){
        fprintf(stderr, "Please enter port number as second argument!\n");
        exit(EXIT_FAILURE);
    }
    int PORT = atoi(argv[1]);
    long num;

```

```

    int sockfd, total, i, rem, arr[20], count = 0, r = 0, result = 0, bin, j,
    ,→ newarr[20], finalarr[20];
    char buffer1[40];
    struct sockaddr_in servaddr;
    if ((sockfd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Socket creation failed!");
        exit(1); }
    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(PORT);
    servaddr.sin_addr.s_addr = inet_addr("127.0.0.1");
    int n, len;
    connect(sockfd, (struct sockaddr*)&servaddr, sizeof(servaddr));
    n = read(sockfd, buffer1, sizeof(buffer1));
    num = atol(buffer1);
    total = countbits(num);
    printf("Received data: %lu\n", num);
    i = 1;
    while(num > 0)
    {
        rem = num % 10;
        arr[total - i] = rem;
        num /= 10;
        i++;
    }
    for(i = 1; i <= total; i++)
        if(ceil(log2(i)) == floor(log2(i)))
            r++;
    int k = 0;
    for(i = 0; i < 4; i++)
    {
        for(j = 1; j <= total; j++)
        {
            bin = binary(j);
            if(ispresent(bin, i + 1))
                count += arr[total - j];
        }
        if(count % 2 == 0)
            result += pow(10, k) * 0;
        else
            result += pow(10, k) * 1;
        k++;
    }
    count=0; }
    int error = decimal(result);
    printf("\nError bit in binary: %d\n", result);
    printf("\nError in bit %d\n", error);
    if(arr[total - error] == 0)

```

```

arr[total - error] = 1;
5

else
    arr[total - error] = 0;
    k = 0;
    printf("\nData after error correction: ");
    for(i = total - 1; i >= 0; i--)
    {
        newarr[k] = arr[i];
    k++; }
    int x = 0;
    for(i = 0; i < k; i++)
    {
        if(ceil(log2(i + 1)) != floor(log2(i + 1)))
        {
            finalarr[x] = newarr[i];
        x++; }
    }
    for(i = x - 1; i >= 0; i--)
        printf("%d", finalarr[i]);
    printf("\n");
return 0; }

```

3. Hamming Code specific functions

```

int countbits(long num)
{
    int r, count = 0;
    while(num > 0)
    {
        num = num / 10;
        count++; }
    return count;
}

int binary(int num)
{
    int bin = 0, r;
    int i = 0;
    while(num > 0)
    {
        r = num % 2;
        bin += r * pow(10, i);
        num /= 2;
        i++;
    }
    return bin; }

int ispresent(int num, int pos)
{

```

```

    int rem;
    for(int i = 0; i < pos; i++)
    {
rem = num % 10;
        num = num / 10;
    }
    if(rem == 1)
        return 1;
6
else
    return 0;
}
int isapower2(int n)
{
    if(ceil(log2(n)) == floor(log2(n)))
        return 1;
    else
        return 0;
}
int decimal(int num)
{
    int rem, i = 0, result;
    while(num > 0)
    {
        rem = num % 10;
        result += pow(2, i) * rem;
        num /= 10;
        i++;
    }
    return result;
}

```

Output:

```
→ ./server 8080
Enter the data to send: 1010101

No.of redundant bits required: 4

Data with redundant bits: 10100101111
Introducing error automatically at bit: 5
Data transmitted is 10100111111
```

```
→ ./client 8080
Received data: 10100111111

Error bit in binary: 101

Error in bit 5

Data after error correction: 1010101
```

Learning Outcomes:

I learned how to implement Hamming Error correction using socket programming.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 13
Simulation of TCP Congestion Control Algorithms

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Congestion Control (Reno and Tahoe) variants in NS2

```
tahoe.tcl
# Create a simulator object
set ns [new Simulator]
# Define different colors
# for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
#Open the NAM trace file
set nf [open taho.nam w]
$ns namtrace-all $nf
set tf [open taho.tr w]
$ns trace-all $tf
#Define a 'finish' procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    #Close the NAM trace file
    close $nf
    close $tf
    #Execute NAM on the trace file
    exec nam taho.nam &
    exec awk -f Throughput.awk taho.tr &
    exit 0 }
```

```
# Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
```

```
# Create links between the nodes
```

```
$ns duplex-link $n0 $n1 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
```

```
# Set Queue Size of link (n2-n3) to 10
$ns queue-limit $n0 $n1 10
```

```
# Give node position (for NAM)
#$ns duplex-link-op $n0 $n2 orient right
$ns duplex-link-op $n0 $n1 orient right-down
$ns duplex-link-op $n2 $n1 orient left-down
```

```
# Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n0 $n1 queuePos 0.5
```

```
# Setup a TCP connection
set tcp [new Agent/TCP]
$tcp set window_ 10 ;# Set the window size (e.g., 10 packets)
$tcp set packetSize_ 1000 ;# Set the packet size (e.g., 1000
bytes)
$tcp set timeout_ 1.0 ;# Set the timeout (e.g., 1.0 seconds)
$ns attach-agent $n0 $tcp
```

```
set sink [new Agent/TCPSink]
$ns attach-agent $n2 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
```

```
# Setup a FTP over TCP connection
```

```
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
```

```
# Schedule events for the CBR and FTP agents
#$ns at 0.1 "$cbr start"
$ns at 0.1 "$ftp start"
$ns at 4.0 "$ftp stop"
```

```
#$ns at 4.5 "$cbr stop"
```

```
# Detach tcp and sink agents
# (not really necessary)
#$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n2
$sink"
```

```
# Call the finish procedure after
# 5 seconds of simulation time
$ns at 5.0 "finish"
```

```
# procedure to plot the congestion window
proc plotWindow {tcpSource outfile} {
```

```
    global ns
    set now [$ns now]
    set cwnd [$tcpSource set cwnd_]
```

```
# the data is recorded in a file called congestion.xg (this can
be plotted # using xgraph or gnuplot. this example uses xgraph
to plot the cwnd_
```

```
    puts $outfile "$now $cwnd"
```

```
    $ns at [expr $now+0.1] "plotWindow $tcpSource $outfile"
}
```

```
set outfile [open "taho.xg" w]
$ns at 0.0 "plotWindow $tcp $outfile"
```

```
# Run the simulation
$ns run
```

reno.tcl

```
#Create a simulator object
```

```
set ns [new Simulator]

#Open the nam file basic1.nam and the variable-trace file
basic1.tr
set namfile [open basic1.nam w]
$ns namtrace-all $namfile

set tracefile [open basic1.tr w]
$ns trace-all $tracefile

#Define a 'finish' procedure
proc finish {} {

    global ns namfile tracefile
    $ns flush-trace
    close $namfile
    close $tracefile

    exit 0 }

#Create the network nodes
set A [$ns node]
set R [$ns node]
set B [$ns node]

#Create a duplex link between the nodes

$ns duplex-link $A $R 10Mb 10ms DropTail
$ns duplex-link $R $B 800Kb 50ms DropTail

# The queue size at $R is to be 7, including the packet being
sent

$ns queue-limit $R $B 7

# some hints for nam
```

```
# color packets of flow 0 red
$ns color 0 Red
$ns duplex-link-op $A $R orient right
$ns duplex-link-op $R $B orient right
$ns duplex-link-op $R $B queuePos 0.5
```

```
# Create a TCP sending agent and attach it to A
set tcp0 [new Agent/TCP/Reno]
# We make our one-and-only flow be flow 0
$tcp0 set class_ 0
```

```
$tcp0 set window_ 100
$tcp0 set packetSize_ 960
$ns attach-agent $A $tcp0
```

```
# Let's trace some variables
$tcp0 attach $tracefile
$tcp0 tracevar cwnd_
$tcp0 tracevar ssthresh_
$tcp0 tracevar ack_
```

```
$tcp0 tracevar maxseq_
```

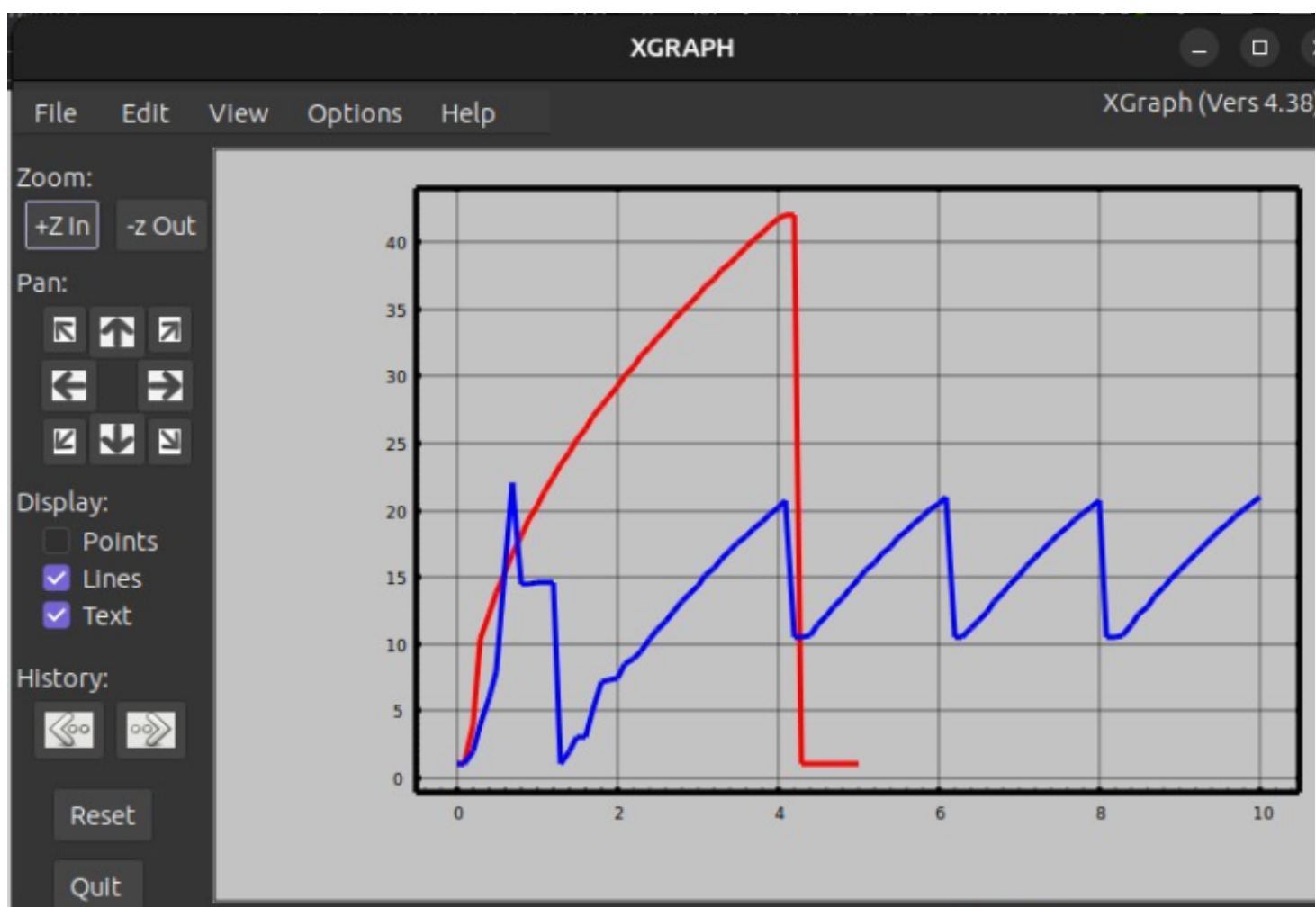
```
#Create a TCP receive agent (a traffic sink) and attach it to B
set end0 [new Agent/TCPSink]
$ns attach-agent $B $end0
```

```
#Connect the traffic source with the traffic sink
$ns connect $tcp0 $end0
```

```
#Schedule the connection data flow; start sending data at T=0,
stop at T=10.0
set myftp [new Application/FTP]
$myftp attach-agent $tcp0
```

```
$ns at 0.0 "$myftp start"
$ns at 10.0 "finish"
```

```
proc plotWindow {tcpSource outfile} {  
    global ns  
  
    set now [$ns now]  
    set cwnd [$tcpSource set cwnd_  
  
    # the data is recorded in a file called congestion.xg (this can  
    be plotted # using xgraph or gnuplot. this example uses xgraph  
    to plot the cwnd_  
  
    puts $outfile "$now $cwnd"  
  
    $ns at [expr $now+0.1] "plotWindow $tcpSource $outfile"  
}  
  
set outfile [open "reno.xg" w]  
$ns at 0.0 "plotWindow $tcp0 $outfile"  
  
#Run the simulation  
$ns run
```



SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 13
Simulation of Distance Vector Routing Protocol

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of **Distance Vector Routing** Protocol

Code:

```
set ns [new Simulator]
```

```
$ns color 1 red $ns color 2 blue
```

```
set n0 [$ns node] set n1 [$ns node] set n2 [$ns node] set n3 [$ns node] set n4  
[$ns node] set n5 [$ns node] set n6 [$ns node] set n7 [$ns node] set n8 [$ns  
node] set n9 [$ns node] set n10 [$ns node] set n11 [$ns node]
```

```
set nr [open out.tr w] $ns trace-all $nr  
set nf [open out.nam w] $ns namtrace-all $nf
```

```
proc finish { } { global ns nr nf $ns flush-trace close $nf
```

```
close $nr  
exec nam out.nam & exit 0  
}
```

```
$ns duplex-link $n0 $n1 $ns duplex-link $n1 $n2 $ns duplex-link $n2 $n3 $ns
```

duplex-link \$n3 \$n4 \$ns duplex-link \$n4 \$n5 \$ns duplex-link \$n5 \$n6 \$ns
duplex-link \$n6 \$n7 \$ns duplex-link \$n7 \$n8

\$ns duplex-link \$n0 \$n8

\$ns duplex-link \$n1 \$n10 1Mb 10ms DropTail \$ns duplex-link \$n0 \$n9 1Mb
10ms DropTail

1Mb 10ms DropTail 1Mb 10ms DropTail 1Mb 10ms DropTail 1Mb 10ms DropTail
1Mb 10ms DropTail 1Mb 10ms DropTail 1Mb 10ms DropTail 1Mb 10ms DropTail

1Mb 10ms DropTail

\$ns duplex-link \$n9 \$n11 1Mb 10ms DropTail \$ns duplex-link \$n10 \$n11 1Mb
10ms DropTail \$ns duplex-link \$n11 \$n5 1Mb 10ms DropTail

\$ns duplex-link-op \$n0 \$n1 orient right
\$ns duplex-link-op \$n1 \$n2 orient down
\$ns duplex-link-op \$n2 \$n3 orient down
\$ns duplex-link-op \$n3 \$n4 orient left
\$ns duplex-link-op \$n4 \$n5 orient left
\$ns duplex-link-op \$n5 \$n6 orient left
\$ns duplex-link-op \$n6 \$n7 orient left
\$ns duplex-link-op \$n7 \$n8 orient up
\$ns duplex-link-op \$n8 \$n0 orient up
\$ns duplex-link-op \$n5 \$n11 orient up

\$ns duplex-link-op \$n0 \$n9 orient right-down \$ns duplex-link-op \$n9 \$n11 orient right

\$ns duplex-link-op \$n11 \$n10 orient right \$ns duplex-link-op \$n1 \$n10 orient left-down

set udp0 [new Agent/UDP] \$ns attach-agent \$n0 \$udp0 \$udp0 set fid_ 1

set cbr0 [new Application/Traffic/CBR] \$cbr0 set packetSize_ 1000
\$cbr0 set rate_ 0.10mb
\$cbr0 set random_ true

\$cbr0 attach-agent \$udp0

set null0 [new Agent/Null] \$ns attach-agent \$n5 \$null0 \$ns connect \$udp0 \$null0

set udp1 [new Agent/UDP] \$ns attach-agent \$n1 \$udp1 \$udp1 set fid_ 2

set cbr1 [new Application/Traffic/CBR] \$cbr1 set packetSize_ 1000
\$cbr1 set rate_ 0.10mb
\$cbr1 set random_ true

\$cbr1 attach-agent \$udp1 set null1 [new Agent/Null]

\$ns attach-agent \$n5 \$null1 \$ns connect \$udp1 \$null1

\$ns rtpproto DV

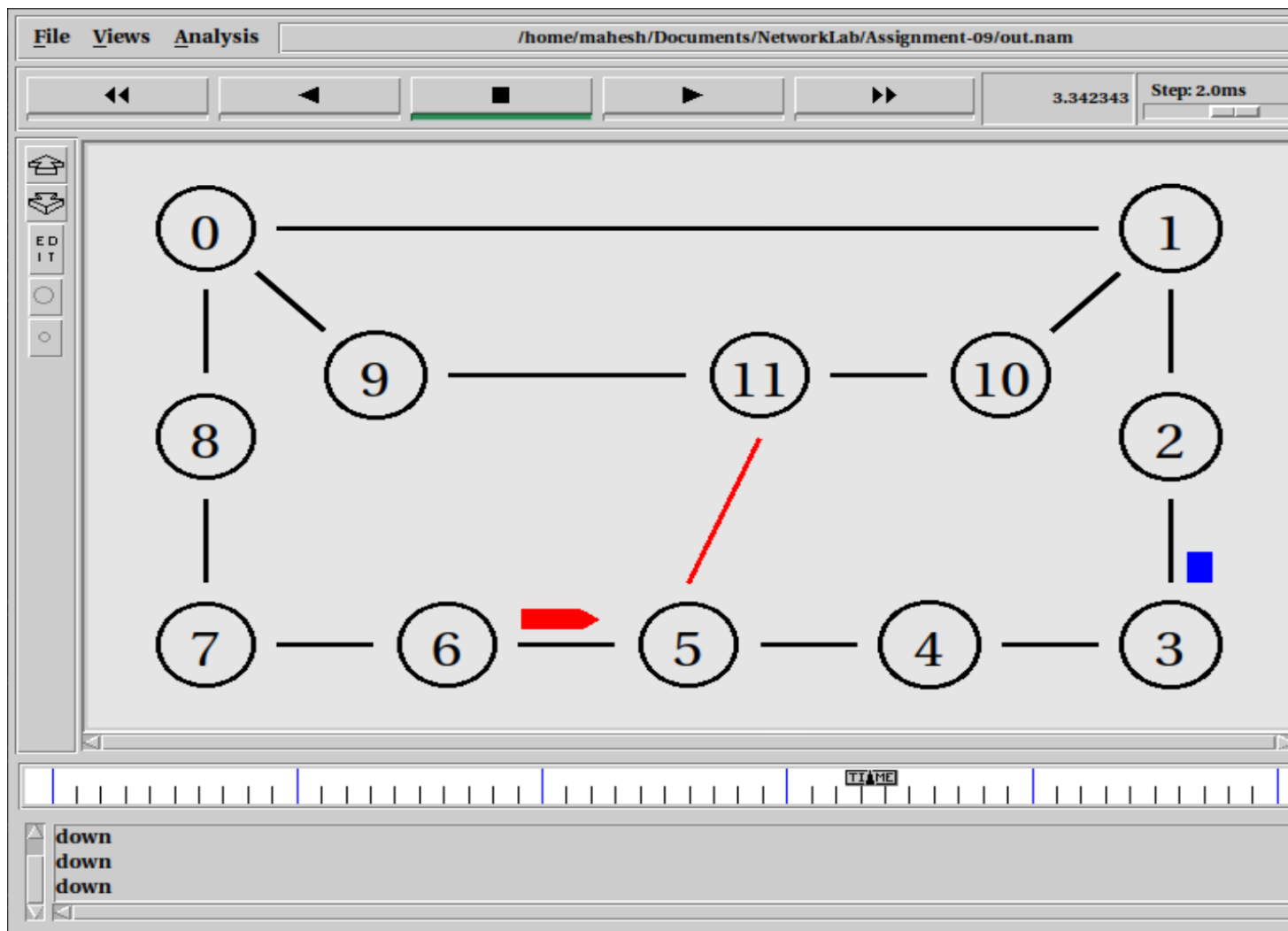
\$ns rtmodel-at 3.0 down \$n11 \$n5 \$ns rtmodel-at 2.0 down \$n7 \$n6 \$ns
rtmodel-at 4.0 up \$n11 \$n5 \$ns rtmodel-at 3.0 up \$n7 \$n6 \$ns at 0.0 "\$cbr0
start"

\$ns at 0.75 "\$cbr1 start"

\$ns at 5.0 "finish"

\$ns run

Output:



Learning Outcomes:

I learned how to implement Distance Vector Routing protocol.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 13
Simulation of TCP Congestion Control Algorithms

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of Congestion Control (Reno and Tahoe) variants in NS2

```
tahoe.tcl
# Create a simulator object
set ns [new Simulator]
# Define different colors
# for data flows (for NAM)
$ns color 1 Blue
$ns color 2 Red
#Open the NAM trace file
set nf [open taho.nam w]
$ns namtrace-all $nf
set tf [open taho.tr w]
$ns trace-all $tf
#Define a 'finish' procedure
proc finish {} {
    global ns nf tf
    $ns flush-trace
    #Close the NAM trace file
    close $nf
    close $tf
    #Execute NAM on the trace file
    exec nam taho.nam &
    exec awk -f Throughput.awk taho.tr &
    exit 0 }
```

```
# Create four nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
```

```
# Create links between the nodes
```

```
$ns duplex-link $n0 $n1 10Mb 10ms DropTail
$ns duplex-link $n1 $n2 2Mb 10ms DropTail
```

```
# Set Queue Size of link (n2-n3) to 10
$ns queue-limit $n0 $n1 10
```

```
# Give node position (for NAM)
#$ns duplex-link-op $n0 $n2 orient right
$ns duplex-link-op $n0 $n1 orient right-down
$ns duplex-link-op $n2 $n1 orient left-down
```

```
# Monitor the queue for link (n2-n3). (for NAM)
$ns duplex-link-op $n0 $n1 queuePos 0.5
```

```
# Setup a TCP connection
set tcp [new Agent/TCP]
$tcp set window_ 10 ;# Set the window size (e.g., 10 packets)
$tcp set packetSize_ 1000 ;# Set the packet size (e.g., 1000
bytes)
$tcp set timeout_ 1.0 ;# Set the timeout (e.g., 1.0 seconds)
$ns attach-agent $n0 $tcp
```

```
set sink [new Agent/TCPSink]
$ns attach-agent $n2 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
```

```
# Setup a FTP over TCP connection
```

```
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
```

```
# Schedule events for the CBR and FTP agents
#$ns at 0.1 "$cbr start"
$ns at 0.1 "$ftp start"
$ns at 4.0 "$ftp stop"
```

```
#$ns at 4.5 "$cbr stop"
```

```
# Detach tcp and sink agents
# (not really necessary)
#$ns at 4.5 "$ns detach-agent $n0 $tcp ; $ns detach-agent $n2
$sink"
```

```
# Call the finish procedure after
# 5 seconds of simulation time
$ns at 5.0 "finish"
```

```
# procedure to plot the congestion window
proc plotWindow {tcpSource outfile} {
```

```
    global ns
    set now [$ns now]
    set cwnd [$tcpSource set cwnd_]
```

```
# the data is recorded in a file called congestion.xg (this can
be plotted # using xgraph or gnuplot. this example uses xgraph
to plot the cwnd_
```

```
    puts $outfile "$now $cwnd"
```

```
    $ns at [expr $now+0.1] "plotWindow $tcpSource $outfile"
}
```

```
set outfile [open "taho.xg" w]
$ns at 0.0 "plotWindow $tcp $outfile"
```

```
# Run the simulation
$ns run
```

reno.tcl

```
#Create a simulator object
```

```
set ns [new Simulator]

#Open the nam file basic1.nam and the variable-trace file
basic1.tr
set namfile [open basic1.nam w]
$ns namtrace-all $namfile

set tracefile [open basic1.tr w]
$ns trace-all $tracefile

#Define a 'finish' procedure
proc finish {} {

    global ns namfile tracefile
    $ns flush-trace
    close $namfile
    close $tracefile

    exit 0 }

#Create the network nodes
set A [$ns node]
set R [$ns node]
set B [$ns node]

#Create a duplex link between the nodes

$ns duplex-link $A $R 10Mb 10ms DropTail
$ns duplex-link $R $B 800Kb 50ms DropTail

# The queue size at $R is to be 7, including the packet being
sent

$ns queue-limit $R $B 7

# some hints for nam
```

```
# color packets of flow 0 red
$ns color 0 Red
$ns duplex-link-op $A $R orient right
$ns duplex-link-op $R $B orient right
$ns duplex-link-op $R $B queuePos 0.5
```

```
# Create a TCP sending agent and attach it to A
set tcp0 [new Agent/TCP/Reno]
# We make our one-and-only flow be flow 0
$tcp0 set class_ 0
```

```
$tcp0 set window_ 100
$tcp0 set packetSize_ 960
$ns attach-agent $A $tcp0
```

```
# Let's trace some variables
$tcp0 attach $tracefile
$tcp0 tracevar cwnd_
$tcp0 tracevar ssthresh_
$tcp0 tracevar ack_
```

```
$tcp0 tracevar maxseq_
```

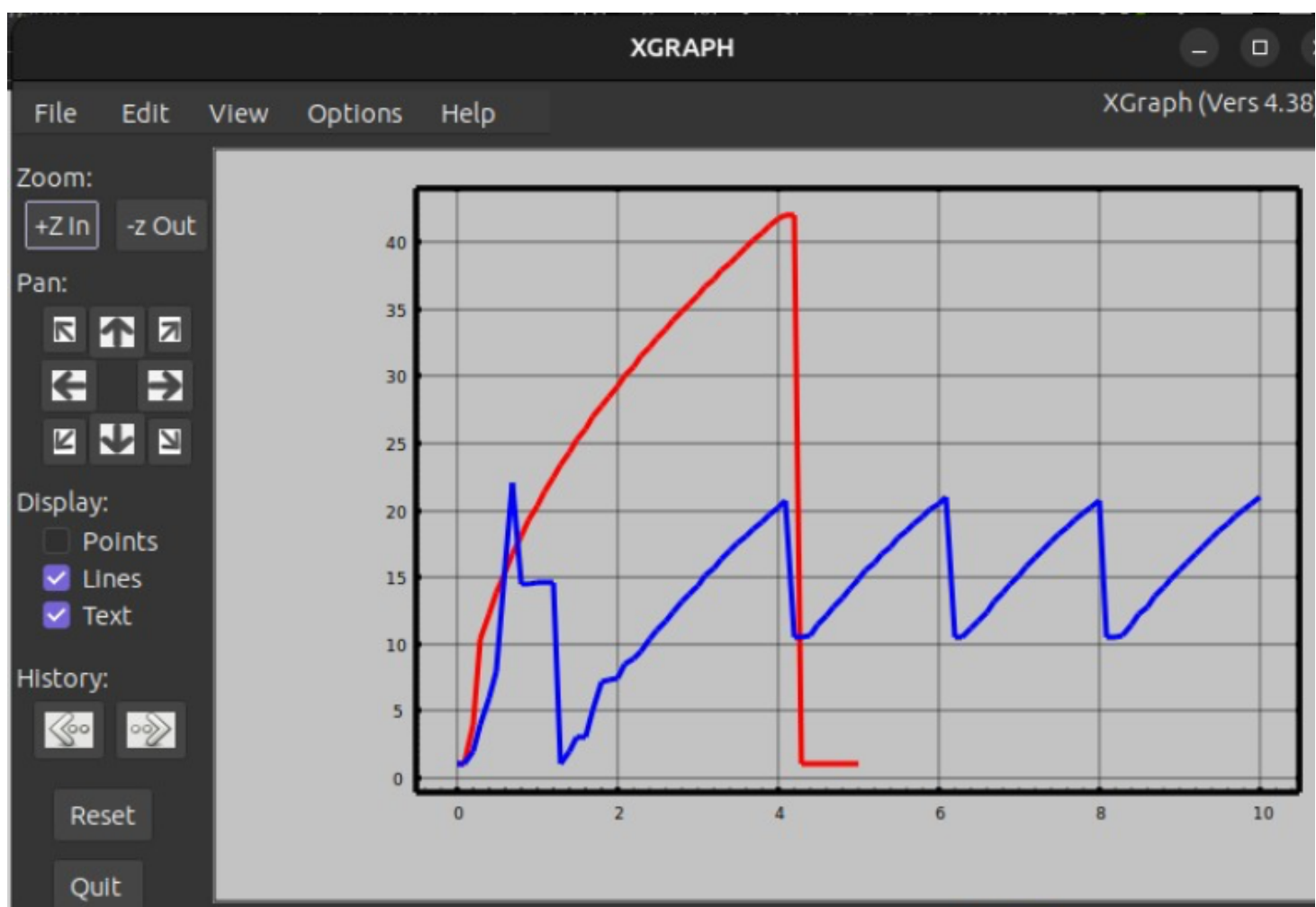
```
#Create a TCP receive agent (a traffic sink) and attach it to B
set end0 [new Agent/TCPSink]
$ns attach-agent $B $end0
```

```
#Connect the traffic source with the traffic sink
$ns connect $tcp0 $end0
```

```
#Schedule the connection data flow; start sending data at T=0,
stop at T=10.0
set myftp [new Application/FTP]
$myftp attach-agent $tcp0
```

```
$ns at 0.0 "$myftp start"
$ns at 10.0 "finish"
```

```
proc plotWindow {tcpSource outfile} {  
    global ns  
  
    set now [$ns now]  
    set cwnd [$tcpSource set cwnd_  
  
    # the data is recorded in a file called congestion.xg (this can  
    be plotted # using xgraph or gnuplot. this example uses xgraph  
    to plot the cwnd_  
  
    puts $outfile "$now $cwnd"  
  
    $ns at [expr $now+0.1] "plotWindow $tcpSource $outfile"  
}  
  
set outfile [open "reno.xg" w]  
$ns at 0.0 "plotWindow $tcp0 $outfile"  
  
#Run the simulation  
$ns run
```



SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 12
TCP UDP Performance Evaluation

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of TCP and UDP protocols and their comparison.

Code

```
#Create a simulator object set ns [new Simulator] #Open the nam trace file set nf
[open out.nam w] $ns namtrace-all $nf

$ns color 1 Blue
$ns color 2 Red
#Define a 'finish' procedure proc finish {} {

global ns nf
$ns flush-trace
#Close the trace file
close $nf
#Execute nam on the trace file exec nam out.nam &
exit 0

}
# Creating Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
```

```

set n4 [$ns node]
set n5 [$ns node]
#Setting Links
$ns duplex-link $n0 $n2 2Mb 10ms DropTail $ns duplex-link $n1 $n2 2Mb 10ms
DropTail $ns duplex-link $n2 $n3 0.3Mb 100ms DropTail $ns duplex-link $n3 $n2
0.3Mb 100ms DropTail $ns duplex-link $n3 $n4 0.5Mb 40ms DropTail $ns
duplex-link $n3 $n5 0.5Mb 40ms DropTail #Setting Topology
$ns duplex-link-op $n0 $n2 orient right-down $ns duplex-link-op $n1 $n2 orient
right-up
$ns duplex-link-op $n2 $n3 orient left-up

$ns duplex-link-op $n3 $n2 orient left
$ns duplex-link-op $n3 $n4 orient up
$ns duplex-link-op $n3 $n5 orient right-up
#Setting Queue Limit
$ns queue-limit $n2 $n3 10
#Setup a TCP connection over 0 and 4 and its flow id, window size, packet size
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$tcp set window_ 8000
$tcp set packetSize_ 552
#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
#Create a UDP agent and attach it to node n0
set udp [new Agent/UDP]
$ns attach-agent $n0 $udp
# Create a CBR traffic source and attach it to udp0
set cbr [new Application/Traffic/CBR]
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 0.01mb
$cbr set random_ false
$cbr attach-agent $udp

```

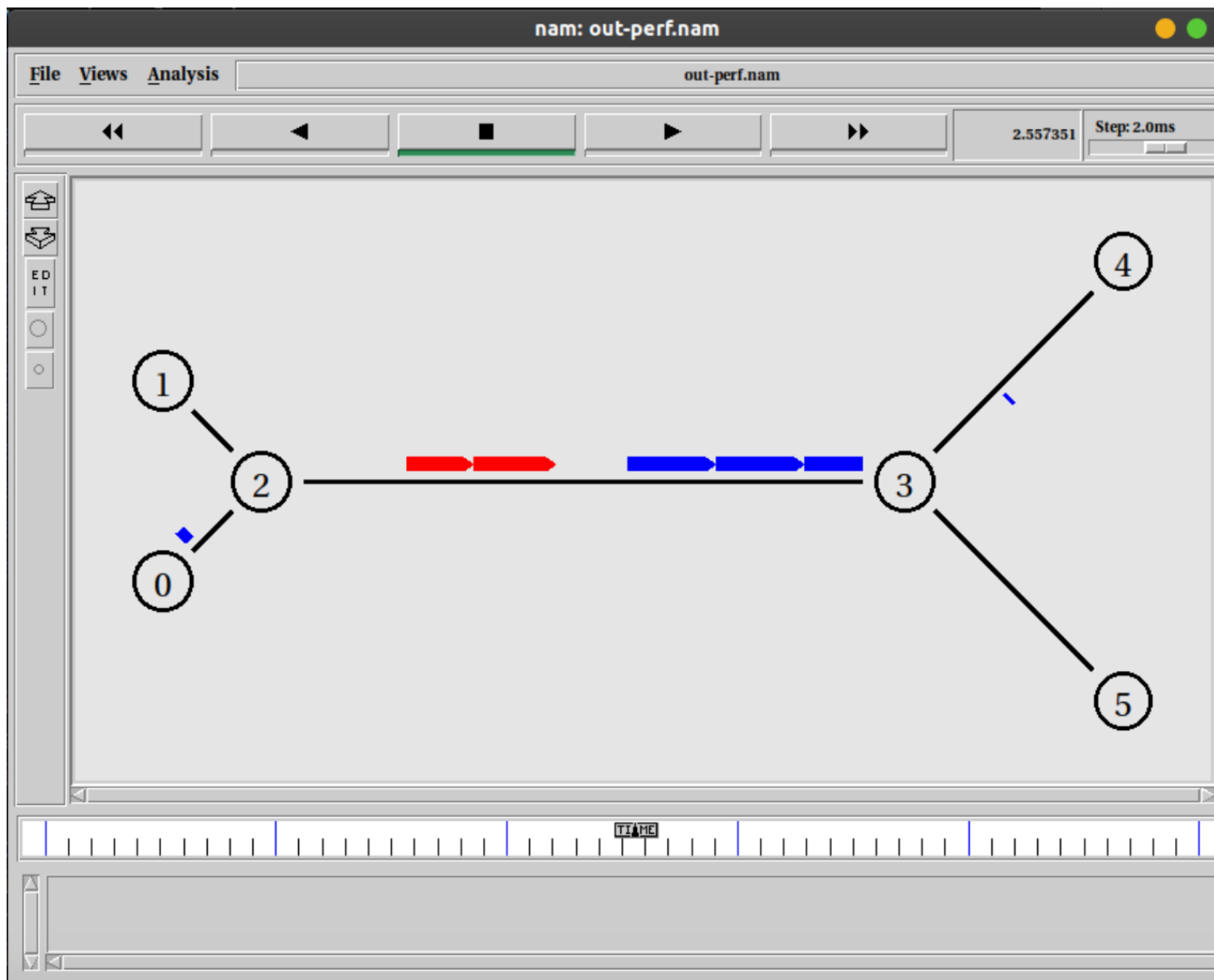
```
#Create a Null agent (a traffic sink) and attach it to node n1 set null [new
Agent/Null]
$ns attach-agent $n5 $null
#Connect the traffic source with the traffic sink
$ns connect $udp $null
#Set Flow ID, Packet Size and Window Size
$udp set fid_ 2
$udp set window_ 8000
$udp set packetSize_ 552
#Start and stop the cbr and ftp

$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.5 "$ftp stop"
$ns at 5.0 "$cbr stop"
#Call the finish procedure after 5 seconds of simulation time $ns at 5.0 "finish"

#Run the simulation

$ns run
```

Output:



Learning Outcomes:

I learned how to compare TCP and UDP protocols.

SSN College of Engineering, Kalavakkam
Department of Computer Science and Engineering
V Semester - CSE 'A'
UCS2511 NETWORKS LAB
Exercise 12
TCP UDP Performance Evaluation

Name : Aathish Sivasubrahmanian 3122215001001

Aim:

Simulation of TCP and UDP protocols and their comparison.

Code

```
#Create a simulator object set ns [new Simulator] #Open the nam trace file set nf
[open out.nam w] $ns namtrace-all $nf

$ns color 1 Blue
$ns color 2 Red
#Define a 'finish' procedure proc finish {} {

global ns nf
$ns flush-trace
#Close the trace file
close $nf
#Execute nam on the trace file exec nam out.nam &
exit 0

}
# Creating Nodes
set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
```

```

set n4 [$ns node]
set n5 [$ns node]
#Setting Links
$ns duplex-link $n0 $n2 2Mb 10ms DropTail $ns duplex-link $n1 $n2 2Mb 10ms
DropTail $ns duplex-link $n2 $n3 0.3Mb 100ms DropTail $ns duplex-link $n3 $n2
0.3Mb 100ms DropTail $ns duplex-link $n3 $n4 0.5Mb 40ms DropTail $ns
duplex-link $n3 $n5 0.5Mb 40ms DropTail #Setting Topology
$ns duplex-link-op $n0 $n2 orient right-down $ns duplex-link-op $n1 $n2 orient
right-up
$ns duplex-link-op $n2 $n3 orient left-up

$ns duplex-link-op $n3 $n2 orient left
$ns duplex-link-op $n3 $n4 orient up
$ns duplex-link-op $n3 $n5 orient right-up
#Setting Queue Limit
$ns queue-limit $n2 $n3 10
#Setup a TCP connection over 0 and 4 and its flow id, window size, packet size
set tcp [new Agent/TCP/Newreno]
$ns attach-agent $n0 $tcp
set sink [new Agent/TCPSink/DelAck]
$ns attach-agent $n4 $sink
$ns connect $tcp $sink
$tcp set fid_ 1
$tcp set window_ 8000
$tcp set packetSize_ 552
#Setup a FTP over TCP connection
set ftp [new Application/FTP]
$ftp attach-agent $tcp
$ftp set type_ FTP
#Create a UDP agent and attach it to node n0
set udp [new Agent/UDP]
$ns attach-agent $n0 $udp
# Create a CBR traffic source and attach it to udp0
set cbr [new Application/Traffic/CBR]
$cbr set type_ CBR
$cbr set packet_size_ 1000
$cbr set rate_ 0.01mb
$cbr set random_ false
$cbr attach-agent $udp

```

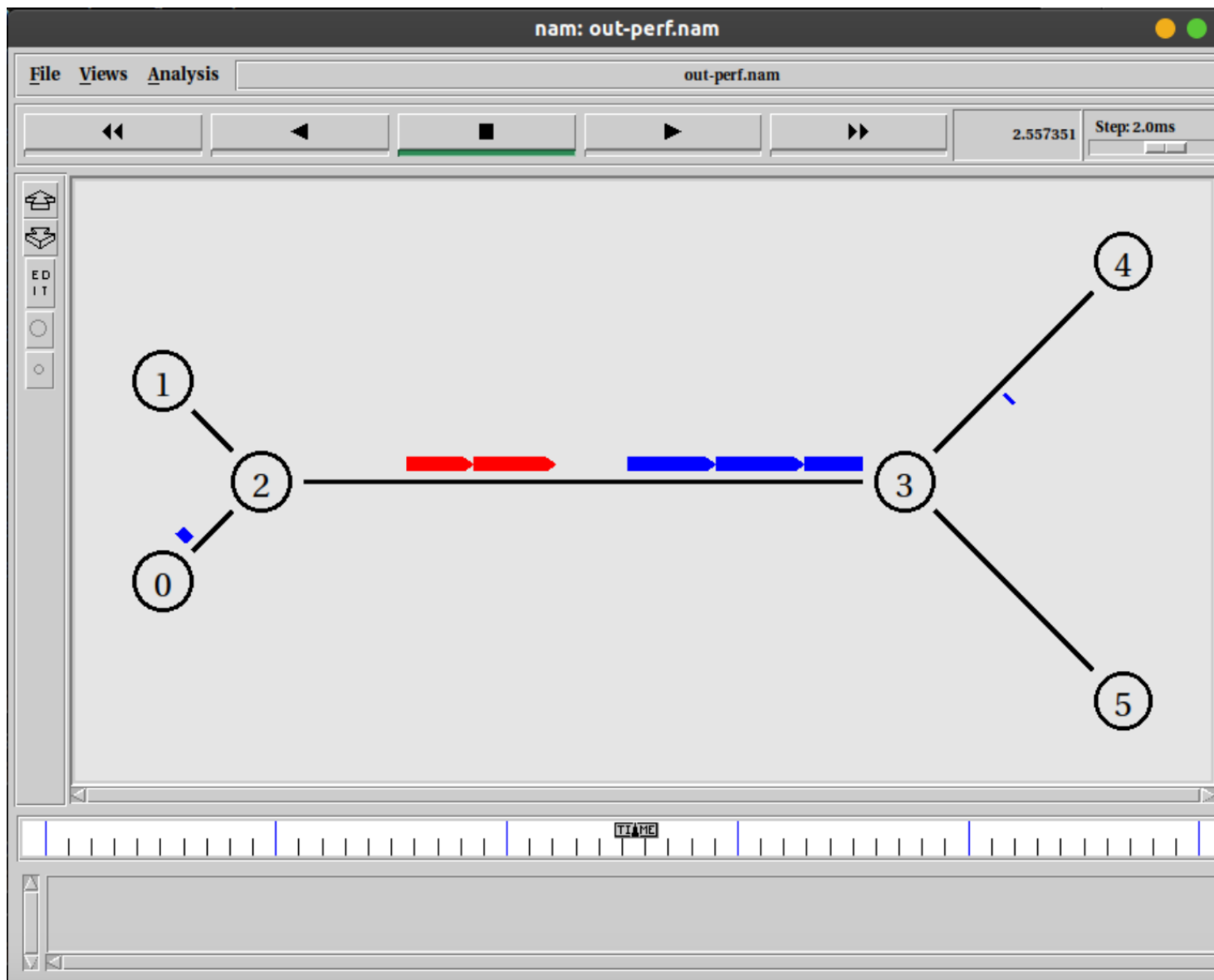
```
#Create a Null agent (a traffic sink) and attach it to node n1 set null [new
Agent/Null]
$ns attach-agent $n5 $null
#Connect the traffic source with the traffic sink
$ns connect $udp $null
#Set Flow ID, Packet Size and Window Size
$udp set fid_ 2
$udp set window_ 8000
$udp set packetSize_ 552
#Start and stop the cbr and ftp

$ns at 0.1 "$cbr start"
$ns at 1.0 "$ftp start"
$ns at 4.5 "$ftp stop"
$ns at 5.0 "$cbr stop"
#Call the finish procedure after 5 seconds of simulation time $ns at 5.0 "finish"

#Run the simulation

$ns run
```

Output:



Learning Outcomes:

I learned how to compare TCP and UDP protocols.

SSN COLLEGE OF ENGINEERING, KALAVAKKAM-603110



Department of Computer Science and Engineering

UCS2501 – Computer Networks

III Year CSE - (V Semester)

Academic Year 2022-23

Batch: 2021- 2025

Mini Project

Implementation of Address Aggregation Concept in C

Faculty In-charge: Dr. N Sujaudeen

Project Students:

Aathish Sivasubrahmanian (3122 21 5001 001)

Achuthanandhan Janarthanan (3122 21 5001 002)

Alamelu Kannan (3122 21 5001 007)

Jahn timer Murali (3122 21 5001 038)

INDEX

S. No.	Contents	Page No.
1.	Problem Definition	
2.	Address Aggregation/ Supernetting: A Brief Overview	
2.1.	Introduction	
2.2.	Supernetting Components and Protocol Requirements	
2.3.	Supernetting Networks: Conditions	
2.4.	Advantages of Supernetting	
3.	C Code implementing Supernetting	
4.	Output Results	
5.	Learning Outcomes	
6.	README.md	
7.	Github Link	

1. Problem Definition

Address aggregation, also known as route aggregation or supernetting, is a networking technique that involves combining multiple contiguous IP address blocks into a larger, more summarized address range. This process helps reduce the size of routing tables in routers and makes the overall routing process more efficient. Address aggregation is a fundamental concept in Internet Protocol (IP) routing and is closely associated with the management of IP address space.

In this project, we aim to simulate address aggregation/ supernetting in C.

2. Address Aggregation/ Supernetting: A Brief Overview

2.1. Introduction

Supernetting is the process of aggregating multiple networks into a single network, essentially the inverse of subnetting. This technique involves combining bits of the network to create larger host addresses, allowing for more efficient allocation of IP addresses at the expense of network addresses. It is commonly used by Internet service providers to optimize IP address usage.

2.2. Supernetting Components and Protocol Requirements

CIDR Value, Network ID, Subnet Mask, Broadcast ID, and Block Size of every route is required for performing supernetting.

- Network ID and broadcast ID are required to check route alignment, and Routes are required to be sequential.
- Block Size is required for the calculation of the summarized route from the routes that are given.
- CIDR value and subnet mask are different notations of a similar thing, and both are used for finding out the number of network bits in the IP address.

Classless Inter-Domain Routing (CIDR) supported Routing Protocols are required for Supernetting.

Interior Gateway Routing Protocol, Routing Information Protocol (RIPv1), and Exterior Gateway Protocol can not send the information needed for Supernetting as these protocols assume classful addressing.

Enhanced Interior Gateway Routing Protocol (EIGRP) is a CIDR-supported classless routing protocol. This protocol summarizes only in the routing table, and then it transmits these routes

that are summarized to its peers. Other CIDR-supported routing protocols may include Open Shortest Path First, EIGRP, RIPv2, IS-IS, and Border Gateway Protocol.

2.3. Supernetting Networks: Conditions

Not all networks are considered suitable for aggregation. The network must follow below given three rules for aggregation.

- Contiguous: All the networks that are to be aggregated must be contiguous.
- Same Size: All the networks must have the same size in the power of 2.
- Divisibility: The ID of the first network must be zero or must be divisible by the block size.

The rules of aggregation can be better comprehended with the help of an example.

Suppose four networks are having IPs 160.70.14.0/26, 160.70.14.64/26, 160.70.14.128/26 and 160.70.14.192/26

. Now let us check if these networks can be aggregated or not.

- Rule 1: Continuity

The provided blocks are continuous as they follow each other without any gaps.

Block 1: 160.70.14.0/26

Block 2: 160.70.14.64/26

Block 3: 160.70.14.128/26

Block 4: 160.70.14.192/26

- Rule 2: Same Size

Each block is of the same size, with a prefix length of /26, which corresponds to 64 IP addresses in each block.

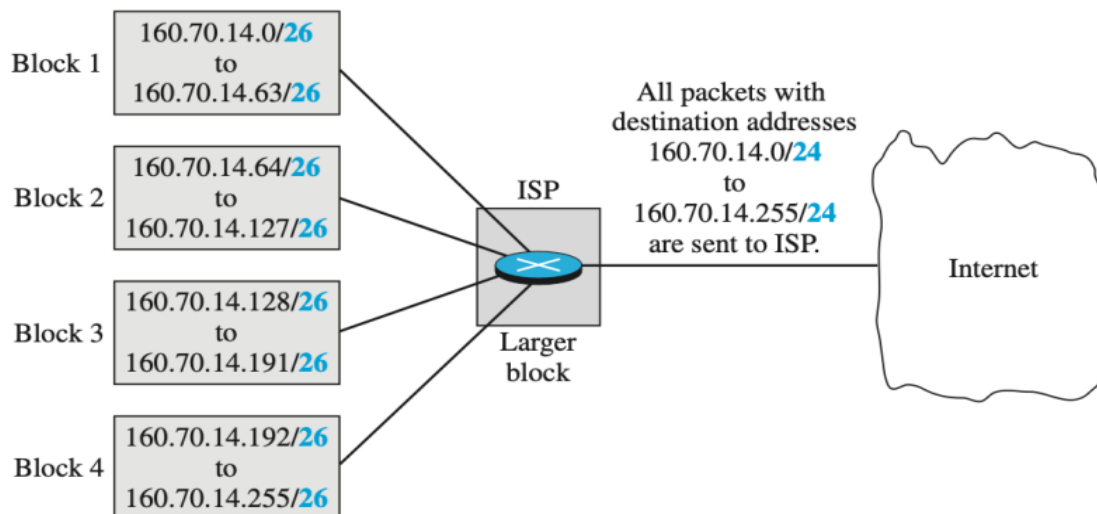
- Rule 3: Divisibility

The size of each block (64 addresses) is divisible by the size of the previous block, satisfying the divisibility rule.

Considering the continuity, same size, and divisibility, these blocks can be supernetted into a larger block:

Supernetted Block: 160.70.14.0/24

The supernetted address covers all the individual blocks and provides a larger address space while maintaining the three rules of supernetting. This example is illustrated visually below



2.4. Advantages

- Supernetting reduces and controls the traffic on the network
- It also helps in solving the lack of IP address problems.
- By Supernetting, entries of routing information are summarized into a single entry of the routing table which results in a decrease in the size of the routing table; hence supernetting saves memory space.

3. Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

```

#define IP_BITS 32
#define MAX_OCTET_VALUE 0xFF

typedef struct {
    uint8_t octets[4];
    int prefix;
} IPAddress;

typedef struct CustomerNode {
    IPAddress startIP;
    IPAddress endIP;
    struct CustomerNode* next;
} CustomerNode;

typedef struct {
    int numCustomers;
    int numAddressesPerCustomer;
} GroupInfo;

GroupInfo getUserInputForGroup(int groupNumber) {
    GroupInfo groupInfo;
    printf("Enter the number of customers for group %d: ",
groupNumber);
    scanf("%d", &groupInfo.numCustomers);
    printf("Enter the number of addresses per customer for group
%d: ", groupNumber);
    scanf("%d", &groupInfo.numAddressesPerCustomer);
    return groupInfo;
}

uint32_t calculateTotalHosts(int prefix) {
    return (1u << (IP_BITS - prefix));
}

uint32_t getNextPowerOfTwo(uint32_t n) {
    uint32_t m = 0;

```

```

    while ((1u << m) < n) {
        m++;
    }
    return m;
}

void printIPAddress(IPAddress ip) {
    printf("%hhu.%hhu.%hhu.%hhu/%d\n", ip.octets[0], ip.octets[1],
ip.octets[2], ip.octets[3], ip.prefix);
}

IPAddress calculateEndIPAddress(IPAddress startIP, int
numAddresses) {
    int newPrefix = IP_BITS - getNextPowerOfTwo(numAddresses);
    uint32_t startAddress = (startIP.octets[0] << 24) |
(startIP.octets[1] << 16) | (startIP.octets[2] << 8) |
startIP.octets[3];
    uint32_t endAddress = startAddress + numAddresses - 1;
    endAddress |= (1u << (IP_BITS - newPrefix)) - 1;

    return (IPAddress) {
        .octets[0] = (endAddress >> 24) & MAX_OCTET_VALUE,
        .octets[1] = (endAddress >> 16) & MAX_OCTET_VALUE,
        .octets[2] = (endAddress >> 8) & MAX_OCTET_VALUE,
        .octets[3] = endAddress & MAX_OCTET_VALUE,
        .prefix = startIP.prefix // Use the original startIP
prefix
    };
}

void printAddressAggregation(IPAddress startIP, IPAddress endIP) {
    printf("Group IP addresses:\n");
    printf("+-----+\n");
    printf("| Start IP: ");
    printIPAddress(startIP);
    printf("| End IP: ");

```

```

    printIPAddress(endIP);
    printf("+-----+\n");
}

CustomerNode* createCustomerNode(IPAddress startIP, IPAddress
endIP) {
    CustomerNode* node = malloc(sizeof(CustomerNode));
    node->startIP = startIP;
    node->endIP = endIP;
    node->next = NULL;
    return node;
}

void printFirstAndLast(CustomerNode* head) {
    int count = 0;
    if (head == NULL) {
        return;
    }

    CustomerNode* current = head;
    printf("Customer IP addresses:\n");
    printf("Customer 1:\n");
    printf("+-----+\n");
    printf("| Start IP: ");
    printIPAddress(current->startIP);
    printf("| End IP:   ");
    printIPAddress(current->endIP);
    printf("+-----+\n");

    while (current->next != NULL) {
        count++;
        current = current->next;
    }
    printf("Customer %d:\n", count + 1);

    printf("+-----+\n");
}

```

```

    printf("| Start IP: ");
    printIPAddress(current->startIP);
    printf("| End IP:   ");
    printIPAddress(current->endIP);
    printf("+-----+\n");
}

IPAddress getNextIPAddress(IPAddress ip) {
    IPAddress nextIP = ip;
    nextIP.octets[3]++;

    if (nextIP.octets[3] == 0) {
        nextIP.octets[2]++;
        if (nextIP.octets[2] == 0) {
            nextIP.octets[1]++;
            if (nextIP.octets[1] == 0) {
                nextIP.octets[0]++;
            }
        }
    }

    return nextIP;
}

IPAddress intToIPAddress(uint32_t value, int prefix) {
    return (IPAddress) {
        .octets[0] = (value >> 24) & MAX_OCTET_VALUE,
        .octets[1] = (value >> 16) & MAX_OCTET_VALUE,
        .octets[2] = (value >> 8) & MAX_OCTET_VALUE,
        .octets[3] = value & MAX_OCTET_VALUE,
        .prefix = prefix
    };
}

uint32_t ipAddressToInt(IPAddress ip) {

```

```

        return (ip.octets[0] << 24) | (ip.octets[1] << 16) |
(ip.octets[2] << 8) | ip.octets[3];
    }

CustomerNode* printCustomerIPAddresses(IPAddress ip, int
prefixLength, int numCustomers, int numAddressesPerCustomer) {
    uint32_t baseAddress = ipAddressToInt(ip);
    CustomerNode* head = NULL;
    CustomerNode* tail = NULL;

    for (int i = 0; i < numCustomers; i++) {
        IPAddress customerIP = intToIPAddress(baseAddress,
prefixLength);

        IPAddress endIP = calculateEndIPAddress(customerIP,
numAddressesPerCustomer);
        CustomerNode* node = createCustomerNode(customerIP, endIP);

        if (head == NULL) {
            head = node;
            tail = node;
        } else {
            tail->next = node;
            tail = node;
        }
        baseAddress += numAddressesPerCustomer;
    }
    return head;
}

int main() {
    IPAddress userIP;
    printf("Enter IP address and prefix: ");
    scanf("%hhu.%hhu.%hhu.%hhu/%d", &userIP.octets[0],
&userIP.octets[1], &userIP.octets[2], &userIP.octets[3],
&userIP.prefix);

```

```

    int numAddresses = calculateTotalHosts(userIP.prefix);
    printIPAddress(userIP);

    int numGroups;
    printf("Enter the number of groups: ");
    scanf("%d", &numGroups);

    printf("\nAddress Aggregations:\n");
    for (int i = 0; i < numGroups; i++) {
        GroupInfo groupInfo = getUserInputForGroup(i + 1);

        int totalAddressesOfGroup = groupInfo.numCustomers *
groupInfo.numAddressesPerCustomer;
        int newPrefix = IP_BITS -
getNextPowerOfTwo(totalAddressesOfGroup);

        IPAddress newIP = calculateEndIPAddress(userIP,
totalAddressesOfGroup);
        userIP.prefix = newPrefix;

        printf("For Group %d:\n", i + 1);
        printAddressAggregation(userIP, newIP);

        CustomerNode* head = printCustomerIPAddresses(userIP,
newPrefix, groupInfo.numCustomers,
groupInfo.numAddressesPerCustomer);
        printFirstAndLast(head);
        printf("=====\n");
        userIP = getNextIPAddress(newIP);
    }
    return 0;
}

```

4. Output

An ISP is granted a block of addresses starting with 190.100.0.0/16 (65,536 addresses). The ISP

needs to distribute these addresses to three groups of customers as follows:

1. The first group has 64 customers; each needs approximately 256 addresses.
2. The second group has 128 customers; each needs approximately 128 addresses.
3. The third group has 128 customers; each needs approximately 64 addresses.

```
Enter IP address and prefix: 190.100.0.0/16
190.100.0.0/16
Enter the number of groups: 3

Address Aggregations:
Enter the number of customers for group 1: 64
Enter the number of addresses per customer for group 1: 256
For Group 1:
Group IP addresses:
+-----+
| Start IP: 190.100.0.0/18
| End IP:   190.100.63.255/16
+-----+
Customer IP addresses:
Customer 1:
+-----+
| Start IP: 190.100.0.0/18
| End IP:   190.100.0.255/18
+-----+
Customer 64:
+-----+
| Start IP: 190.100.63.0/18
| End IP:   190.100.63.255/18
+-----+
=====
Enter the number of customers for group 2: 128
Enter the number of addresses per customer for group 2: 128
For Group 2:
Group IP addresses:
+-----+
| Start IP: 190.100.64.0/18
| End IP:   190.100.127.255/16
+-----+
Customer IP addresses:
Customer 1:
+-----+
| Start IP: 190.100.64.0/18
| End IP:   190.100.64.127/18
+-----+
Customer 128:
+-----+
| Start IP: 190.100.127.128/18
| End IP:   190.100.127.255/18
+-----+
=====
Enter the number of customers for group 3: 128
Enter the number of addresses per customer for group 3: 64
For Group 3:
Group IP addresses:
+-----+
| Start IP: 190.100.128.0/19
| End IP:   190.100.159.255/16
+-----+
Customer IP addresses:
Customer 1:
+-----+
| Start IP: 190.100.128.0/19
| End IP:   190.100.128.63/19
+-----+
Customer 128:
+-----+
| Start IP: 190.100.159.192/19
| End IP:   190.100.159.255/19
+-----+
=====
```

5. Learning Outcomes

- Understood how IP addresses are allocated
- Learned how supernetting optimizes utilization of IP Addresses, and size of routing tables
- Understood CIDR and Classless Routing

6. README.md

☰ README.md



IP Address Aggregation Project

Overview

This C project focuses on IP address aggregation for efficient utilization of address space. It allows users to input a starting IP address and prefix, and then defines address aggregations for multiple groups based on user-specified parameters such as the number of customers per group and addresses per customer.

Project Structure

The project consists of a single C file:

- `ip_aggregation.c`: This file contains the main implementation of the IP address aggregation algorithm. It includes functions for calculating total hosts, finding the next power of two, printing IP addresses, calculating end IP addresses, creating customer nodes, and aggregating and printing customer IP addresses.

Getting Started

1. Clone the repository to your local machine:

```
git clone https://github.com/alameluk17/AddressAggregation.git
```



2. Compile the C file:

```
gcc ip_aggregation.c -o ip_aggregation
```



3. Run the executable:

```
./ip_aggregation
```



4. Follow the on-screen prompts to input the starting IP address, prefix, and group details.

Usage

The program prompts the user to enter the starting IP address and prefix. It then iterates over multiple groups, allowing the user to specify the number of customers and addresses per customer for each group. The program calculates and displays the address aggregation for each group, along with the corresponding customer IP addresses.

7. GitHub Link -

<https://github.com/alameluk17/AddressAggregation>